Algoritmi e Protocolli per la Sicurezza

Project Work

WP1: Modello

E-Vote: Protocollo per il voto sicuro

Gruppo 16: Gennaro Foschillo, Francesco Vittorio Allocca

A.A. 2025–2026

Indice

1 Scenario del progetto e Obiettivi **1**

2 Obiettivo del Work Package **1**

3 Descrizione delle funzionalità del sistema **1**

4 Modellazione ad Alto Livello delle componenti **2**

4.1 Attori Principali 2

4.2 Modellazione del sistema 4

5 Assunzioni di fiducia **5**

5.1 Assunzioni generali del modello 6

6 Proprietà di Sicurezza da preservare **7**

7 Revocabilità e sostituzione del voto **8**

8 Threat Models **9**

8.1 Elettore disonesto 9

8.2 Insider all'interno del Voting Server / Avversario interno privilegiato . . . 10

8.3 Registro Elettorale compromesso o disonesto 11

8.4 Voting Client compromesso o disonesto 12

8.5 Attaccante esterno 13

8.6 Government Authority Disonesta 14

8.7 Scrutiny Authority compromessa o disonesta 16

9 Priorità, Compromessi e Rischi Residui **17**

1 Scenario del progetto e Obiettivi

Nei sistemi democratici moderni, la conduzione di elezioni sicure, trasparenti e verificabili è un requisito fondamentale. Il voto elettronico, inteso come la raccolta e il conteggio di voti tramite sistemi digitali, promette di aumentare l'accessibilità, ridurre i costi e accelerare lo scrutinio. Tuttavia, la sua adozione solleva criticità di sicurezza profonde:

- Violazione della segretezza del voto;
- Manipolazione o alterazione dei voti espressi;
- Impossibilità per il singolo elettore di verificare che il proprio voto sia stato conteggiato correttamente;
- Attacchi alla disponibilità del sistema durante le operazioni di voto;
- Coercizione o acquisto di voto, facilitati dalla mancanza di un ambiente fisicamente controllato.

Un protocollo ideale per il voto elettronico deve garantire al contempo la segretezza del voto, la verificabilità del risultato e la resistenza a manipolazioni, anche da parte degli stessi gestori del sistema.
L'obiettivo di questo progetto è dunque la realizzazione di un protocollo di voto elettronico sicuro, considerando il contesto di un'elezione di tipo referendum binario, in cui ogni elettore può esprimere una sola preferenza tra due opzioni possibili. La scelta di questo contesto riduce notevolmente la complessità progettuale del modello, permettendo quindi di concentrarsi sulle proprietà fondamentali per un sistema di voto sicuro, ovvero: autenticità, unicità del voto, segretezza, integrità, verificabilità individuale e verificabilità universale.

2 Obiettivo del Work Package

Questo work package si occuperà di definire il modello che si intende realizzare, fornendone una descrizione ad alto livello. Verranno definiti i vari attori onesti del sistema, ovvero le entità coinvolte per la corretta realizzazione del sistema ed il suo utilizzo.
Successivamente, verranno definiti i Threat Model, ovvero le entità che potenzialmente possono compromettere il corretto funzionamento del sistema.
Infine, vengono definite le proprietà del sistema che si intende preservare in presenza di attacchi da parte dei Threat Model.

3 Descrizione delle funzionalità del sistema

La funzionalità che si intende realizzare è un sistema per la conduzione di un'elezione elettronica sicura, in cui un avente diritto al voto possa: autenticarsi, ottenere l'autorizzazione al voto, esprimere una sola preferenza al voto, inviare la scheda elettorale ed assicurarsi che il proprio voto sia stato correttamente incluso nel conteggio finale. Il sistema deve infine permettere la pubblicazione di un risultato finale verificabile, riducendo la fiducia

richiesta verso un'unica entità centrale.
Per questo motivo, il sistema viene modellato separando logicamente le funzioni di autenticazione e autorizzazione dalle funzioni di raccolta del voto, registrazione e scrutinio. Nel modello proposto, autenticazione dell'elettore e verifica dell'idoneità sono svolte dalla Government Authority, ma la raccolta della scheda rimane affidata a una componente distinta. Il principale motivo di questa scelta è ridurre il rischio che l'entità che conosce l'identità reale dell'elettore possa conoscere anche la preferenza espressa.

4 Modellazione ad Alto Livello delle componenti

In questa sezione vengono presentati gli attori principali coinvolti nel modello di voto elettronico proposto e i rispettivi obiettivi. Successivamente viene descritta, ad alto livello, l'interazione tra tali attori durante le diverse fasi dell'elezione: autenticazione dell'elettore, verifica dell'idoneità, autorizzazione al voto, raccolta della scheda, registrazione su un registro verificabile e scrutinio finale.

L'obiettivo della modellazione è chiarire quali responsabilità sono assegnate a ciascuna entità e quali informazioni ciascuna di esse può conoscere e quali invece non deve conoscere.

4.1 Attori Principali

- **Voting Client**:
  Descrizione: interfaccia applicativa attraverso cui l'elettore interagisce con il sistema di voto. Permette di avviare il flusso di autenticazione, ottenere l'accesso alla fase di voto, compilare la scheda e inviarla al sistema.
  Obiettivo: consentire all'elettore di esprimere la propria preferenza e di ricevere le informazioni necessarie per verificare l'inclusione della scheda nel registro.
  Conoscenza: il Client può trattare temporaneamente la preferenza selezionata dall'elettore e le informazioni necessarie all'interazione con il sistema. Tuttavia, nel modello proposto, quest'entità non deve conservare stabilmente dati che permettano di collegare identità reale, autorizzazione e voto espresso. Poiché viene eseguito sul dispositivo dell'utente, non viene considerato una componente pienamente fidata.
- **Elettore Onesto**:
  Descrizione: persona fisica avente diritto al voto che esprime una sola preferenza valida.
  Obiettivo: autenticarsi correttamente, ottenere l'autorizzazione al voto ed esprimere una preferenza di voto.
  Conoscenza: l'elettore onesto conosce la propria identità reale e la propria preferenza di voto. Una volta inviata la scheda elettorale, non tenta di votare più volte, non ripudia la scelta fatta e non cerca di alterare il corretto funzionamento del sistema.
- **Government Authority**:
  Descrizione: è l'entità che si occupa di identificare e autenticare l'elettore, al fine di verificare la sua idoneità alla partecipazione all'elezione.
  Obiettivo: garantire la partecipazione all'elezione solo agli elettori legittimi e rilasciare per ogni elettore un'autorizzazione che gli permetterà di esercitare il voto.

Inoltre, dispone di un modulo interno per autenticare l'elettore tramite servizi di identità digitale, come ad esempio SPID o CIE.
Conoscenza: quest'entità conosce l'identità reale dell'elettore e lo stato di autorizzazione al voto, ma non deve conoscere la sua preferenza espressa.

- **Voting Server**:
  Descrizione: è l'entità incaricata di ricevere le schede elettorali dagli elettori autorizzati e di verificare che ciascun voto provenga da un soggetto abilitato.
  Obiettivo: controllare la validità dell'autorizzazione al voto, impedire l'utilizzo duplicato della stessa credenziale e accettare soltanto voti conformi alle regole dell'elezione. Una volta validato il voto, il Voting Server lo inoltra al Registro Elettorale per la sua registrazione in forma verificabile.
  Conoscenza: il Voting Server non dovrebbe conoscere la vera identità dell'elettore, ma soltanto le informazioni necessarie a stabilire che il voto proviene da un soggetto autorizzato. Inoltre, non dovrebbe conoscere la preferenza in chiaro, ma gestire soltanto una rappresentazione protetta del voto, ad esempio un voto cifrato.

- **Registro Elettorale**:
  Descrizione: è l'entità logica incaricata di conservare i voti validamente accettati dal Voting Server e di renderli pubblicamente verificabili. Nel modello proposto, il Registro Elettorale rappresenta un registro di sola scrittura, cioè una struttura in cui i voti registrati non possono essere modificati, rimossi o sostituiti senza che tale alterazione risulti rilevabile.
  Obiettivo: supportare la verificabilità individuale e universale dell'elezione. In particolare, ogni elettore deve poter controllare che il proprio voto sia stato incluso nel registro, mentre osservatori esterni devono poter verificare che il risultato finale sia coerente con l'insieme dei voti registrati.
  Conoscenza: il Registro Elettorale non deve conoscere l'identità reale degli elettori né la preferenza in chiaro, ma deve conservare solamente informazioni pubblicabili relative ai voti.

- **Scrutiny Authority**:
  Descrizione: è l'entità logica incaricata di eseguire lo scrutinio finale a partire dai dati registrati nel Registro Elettorale. Nel modello proposto viene considerata come un insieme di nodi fiduciari che cooperano nella fase di conteggio. Questa distribuzione ha l'obiettivo di ridurre il rischio che una singola entità possa controllare autonomamente l'accesso alla fase di scrutinio o manipolare il risultato. La distribuzione di questa entità è puramente organizzativa: il suo scopo è separare i privilegi di scrutinio, non di implementare una separazione crittografica.
  Conoscenza: la Scrutiny Authority accede ai dati registrati nel Registro Elettorale e alle informazioni necessarie per eseguire lo scrutinio finale. Nel modello non deve conoscere l'identità reale degli elettori e non deve poter associare una preferenza a uno specifico votante. I singoli nodi della Scrutiny Authority non devono disporre, individualmente, di informazioni sufficienti per collegare identità e preferenza.

- **Osservatore pubblico / Verificatore esterno**:
  Descrizione: soggetto esterno al processo elettorale che consulta i dati pubblicati dal sistema al termine della votazione, con l'obiettivo di verificare la correttezza del risultato finale. Può rappresentare un cittadino, un membro della comunità universitaria, un osservatore indipendente o qualsiasi parte interessata alla trasparenza

dell'elezione.
Obiettivo: controllare che il risultato pubblicato sia coerente con l'insieme delle schede registrate nel Registro Elettorale e con le informazioni pubbliche rese disponibili dal sistema. In particolare, deve poter verificare che il conteggio finale sia compatibile con i voti pubblicati, senza dover accedere a informazioni riservate sugli elettori.
Conoscenza: l'osservatore pubblico conosce esclusivamente i dati pubblici dell'elezione, come i record pubblicati nel Registro Elettorale, eventuali riferimenti crittografici, prove di inclusione e risultato finale. Non conosce l'identità reale degli elettori, non accede ai dati interni della Government Authority o del Voting Server e non deve poter associare una preferenza a uno specifico votante.

4.2 Modellazione del sistema

Il funzionamento del sistema viene modellato come una sequenza di fasi logiche, ciascuna associata a una responsabilità distinta. L'obiettivo di questa separazione è ridurre la fiducia richiesta verso una singola entità centrale e limitare la possibilità che identità reale dell'elettore, autorizzazione al voto e preferenza espressa siano associate direttamente.

1. **Identificazione e autenticazione dell'elettore presso la Government Authority.**
   Il potenziale elettore accede al sistema tramite il Voting Client e avvia la fase di identificazione presso la Government Authority. In questa fase viene verificata l'identità reale dell'utente, senza trattare informazioni relative alla preferenza di voto. La funzione di autenticazione è logicamente distinta dalle successive fasi di autorizzazione e raccolta della scheda.

2. **Verifica dell'idoneità da parte della Government Authority.**
   Dopo il completamento dell'autenticazione, la Government Authority verifica che l'utente autenticato sia incluso tra gli aventi diritto per l'elezione considerata. Questa fase distingue l'autenticazione dell'identità dall'autorizzazione al voto: un utente può essere correttamente identificato, ma non necessariamente abilitato a partecipare a quella specifica elezione.

3. **Rilascio dell'autorizzazione al voto.**
   Se l'utente risulta idoneo, la Government Authority rilascia un'autorizzazione al voto. Nel modello, tale autorizzazione deve consentire al Voting Server di verificare che la scheda provenga da un soggetto abilitato, senza ricevere l'identità reale dell'elettore durante la fase di raccolta del voto. L'autorizzazione non deve inoltre essere direttamente associabile alla preferenza che sarà successivamente espressa.

4. **Compilazione della scheda tramite Voting Client.**
   L'elettore utilizza il Voting Client per compilare la scheda elettorale, scegliendo una delle opzioni disponibili nel referendum binario. Il Client ha il compito di mostrare correttamente le alternative previste e permettere l'invio della scheda. Nel modello, il Client non deve conservare informazioni che permettano di collegare stabilmente identità reale, autorizzazione e voto espresso.

5. **Invio della scheda al Voting Server e verifica.**
   La scheda viene inviata al Voting Server insieme alle informazioni necessarie a

dimostrare l'autorizzazione al voto. Il Voting Server verifica che l'autorizzazione sia valida e che non sia già stata utilizzata per esprimere un voto valido. Questa fase è necessaria per preservare autenticità e unicità del voto. Le schede non valide, duplicate o non conformi alle regole dell'elezione non vengono accettate come voti validi.

6. **Registrazione del voto valido nel Registro Elettorale.**
   Le schede accettate dal Voting Server vengono registrate nel Registro Elettorale, modellato come registro append-only. Tale registro deve rendere rilevabile ogni tentativo successivo di alterazione, rimozione o sostituzione dei voti registrati. A seguito della registrazione, l'elettore ottiene una ricevuta o prova di inclusione che gli consente di verificare la presenza della propria scheda nel registro, senza costituire una prova pubblicamente utilizzabile della preferenza espressa.

7. **Chiusura delle urne.**
   Al termine prestabilito dell'elezione, il sistema non accetta ulteriori schede elettorali. La chiusura delle urne separa la fase di raccolta dalla fase di scrutinio e consente di definire in modo chiaro l'insieme dei voti validi da considerare nel conteggio finale.

8. **Scrutinio finale da parte della Scrutiny Authority e pubblicazione dei risultati.**
   Dopo la chiusura delle urne, la Scrutiny Authority elabora le schede registrate nel Registro Elettorale ed esegue il conteggio finale secondo le regole dell'elezione. Il risultato deve essere pubblicato in forma trasparente e verificabile, in modo che osservatori esterni possano controllare la coerenza tra i voti registrati e il risultato dichiarato. Nel modello, lo scrutinio deve avvenire senza consentire l'associazione diretta tra identità reale dell'elettore e preferenza espressa.

5 Assunzioni di fiducia

Il modello separa le responsabilità tra più entità: autenticazione, autorizzazione, raccolta del voto, registrazione e scrutinio. Tale separazione non elimina la necessità di fidarsi parzialmente di alcune componenti, ma riduce il rischio che una singola entità possa associare direttamente identità reale dell'elettore e preferenza espressa.

Si assume che gli aventi diritto siano già registrati presso un servizio di identificazione digitale riconosciuto, come SPID, CIE o un servizio equivalente. Il modello non include quindi una fase autonoma di registrazione dell'identità digitale, ma delega tale responsabilità a un'infrastruttura esterna.

- **Voting Client**: si assume che mostri correttamente le opzioni previste per il referendum binario e che trasmetta la scheda secondo il flusso previsto dal sistema. Tuttavia, essendo eseguito sul dispositivo dell'elettore, non viene considerato pienamente fidato. Il rischio residuo è che un client compromesso possa osservare la preferenza, modificarla prima dell'invio o conservare dati utili a collegare identità, autorizzazione e voto.
- **Government Authority**: si assume che identifichi e autentichi correttamente gli utenti, verifichi correttamente l'idoneità degli elettori e rilasci autorizzazioni solo

agli aventi diritto. Questa entità deve conoscere l'identità reale dell'elettore per controllarne l'idoneità, ma non deve conoscere la preferenza espressa e non deve partecipare alla raccolta della scheda. Il rischio residuo è che una Government Authority malevola possa autenticare utenti non legittimi, negare l'accesso a utenti legittimi, rilasciare autorizzazioni indebite, duplicare autorizzazioni o conservare metadati temporali utili alla correlazione con la fase di voto.

- **Voting Server**: si assume che verifichi la validità delle autorizzazioni ricevute, impedisca il riutilizzo della stessa autorizzazione e accetti solo schede conformi alle regole dell'elezione. Non è considerato pienamente fidato rispetto alla segretezza del voto: nel modello non deve conoscere l'identità reale dell'elettore né la preferenza in chiaro. Il rischio residuo è che un Voting Server malevolo possa rifiutare voti validi, accettare voti non validi, ritardare la registrazione o conservare metadati utili alla correlazione.
- **Registro Elettorale**: si assume che si comporti come un registro di sola scrittura, rendendo rilevabili modifiche, cancellazioni o sostituzioni successive dei voti registrati. Non viene considerato pienamente fidato rispetto alla disponibilità o alla corretta pubblicazione dei dati. Il rischio residuo è che un registro compromesso possa ritardare la pubblicazione, omettere informazioni o rendere più difficile la verifica individuale e universale.
- **Scrutiny Authority**: si assume che esegua il conteggio secondo le regole del referendum binario e pubblichi le informazioni necessarie alla verifica del risultato. Non viene considerata pienamente fidata: una Scrutiny Authority malevola potrebbe pubblicare un risultato scorretto o incompleto. La verificabilità universale ha lo scopo di rendere rilevabili discrepanze tra voti registrati e risultato finale.

5.1 Assunzioni generali del modello

Si assume che la Government Authority abbia accesso a un'infrastruttura di identificazione digitale riconosciuta, in grado di verificare l'identità reale degli utenti.

Si assume inoltre che non vi sia collusione totale tra le componenti principali del sistema. In particolare, una collusione tra Government Authority, Voting Server, Registro Elettorale e Scrutiny Authority potrebbe compromettere lo pseudoanonimato, poiché consentirebbe di collegare identità reale, autorizzazione e scheda registrata.

Anche collusioni parziali tra componenti diverse possono però indebolire alcune proprietà del sistema. Nel modello si considerano quindi rilevanti i seguenti casi:

- **Government Authority e Voting Server**: una collaborazione tra queste due entità potrebbe facilitare la correlazione tra identità reale, rilascio dell'autorizzazione e utilizzo della stessa durante la fase di voto, soprattutto tramite informazioni temporali o log operativi.
- **Voting Server e Registro Elettorale**: una collaborazione tra queste due componenti potrebbe rendere più difficile rilevare omissioni, ritardi o registrazioni scorrette delle schede accettate, indebolendo integrità e verificabilità individuale.

- **Registro Elettorale e Scrutiny Authority**: una collaborazione tra registro e autorità di scrutinio potrebbe influenzare la fase di conteggio o la pubblicazione del risultato finale, soprattutto se non sono disponibili dati pubblici sufficienti per la verifica universale.
- **Government Authority, Voting Server e Scrutiny Authority**: una collusione tra queste entità rappresenta uno scenario particolarmente critico, perché potrebbe combinare identità reale, autorizzazione al voto e informazioni ricavate durante lo scrutinio, riducendo fortemente lo pseudoanonimato.

Tali collusioni non vengono considerate impossibili, ma rappresentano rischi residui del modello. La separazione delle responsabilità non elimina questi rischi, ma riduce la quantità di informazioni disponibile a ciascuna singola componente e rende necessario il coinvolgimento coordinato di più parti per compromettere le proprietà principali del sistema.

Infine, si assume che gli avversari considerati non siano in grado di rompere primitive crittografiche correttamente implementate, ma possano sfruttare errori logici, privilegi interni, correlazioni temporali, canali di comunicazione o compromissioni locali, come verrà specificato successivamente.

6 Proprietà di Sicurezza da preservare

Il sistema descritto, mira dunque a preservare le seguenti proprietà in presenza di attacchi:

1. **Pseudoanonimato**
   Nessuna singola entità, inclusa la Government Authority, deve disporre contemporaneamente di identità reale e preferenza. La GA conosce identità e autorizzazionem non la preferenza. Le altre entità trattano schede o dati di scrutinio, ma non identità reali.
2. **Integrità:**
   I voti inviati non devono in alcun modo essere modificati, soppressi o introdotti in maniera fraudolenta.
3. **Verificabilità individuale:**
   Il singolo elettore deve poter verificare che il proprio voto sia stato registrato correttamente e incluso nel conteggio.
4. **Verificabilità universale:**
   Osservatori esterni devono poter assicurarsi che il referendum sia stato svolto correttamente, senza manomissioni.
5. **Responsabilità dell'emissione delle autorizzazioni:**
   le autorizzazioni al voto emesse dalla Government Authority devono essere tracciabili rispetto all'autorità emittente. In caso di emissione errata o fraudolenta, l'autorità non deve poter negare l'avvenuta emissione. Questa proprietà riguarda l'accountability della fase di autorizzazione e non deve trasformarsi in una prova pubblica della preferenza espressa dall'elettore.

6. **Efficienza:**
   Il modello considera l'efficienza come vincolo progettuale: le operazioni previste devono essere compatibili con un numero elevato di elettori e non devono richiedere costi computazionali sproporzionati per client, server e verificatori

7. **Autenticità:**
   Il requisito di autenticità stabilisce che l'ottenimento dell'autorizzazione al voto, con conseguente accesso alla piattaforma di voto, siano riservati esclusivamente agli elettori la cui identità sia stata preventivamente verificata e autenticata dalla Government Authority.

8. **Confidenzialità dei dati personali:**
   le informazioni identificative dell'elettore devono essere accessibili solo alle entità che ne hanno bisogno per autenticazione e autorizzazione, e non devono essere propagate alle componenti incaricate della raccolta e dello scrutinio.

9. **Segretezza del voto:**
   la preferenza espressa dal singolo elettore non deve essere rivelata a soggetti non autorizzati e non deve essere osservabile in chiaro durante raccolta e registrazione. Durante lo scrutinio può essere trattata solo dalla procedura autorizzata, senza informazioni che permettano l'associazione identità-voto.

10. **Disponibilità**: il sistema deve rimanere utilizzabile durante il periodo di voto e deve ridurre il rischio che attacchi o malfunzionamenti impediscano a elettori legittimi di partecipare.

11. **Unicità del voto:**
    La proprietà di unicità richiede che ciascun elettore possa esprimere al più un voto valido. Di conseguenza, non deve essere possibile che un elettore possa inviare più di una scheda elettorale valida.

7 Revocabilità e sostituzione del voto

Nel modello non si consente all'elettore di modificare o sostituire il voto dopo la registrazione della scheda nel Registro Elettorale. Questa scelta semplifica il controllo di unicità e riduce la superficie di attacco legata a replay, contestazioni fraudolente e gestione di più versioni della stessa scheda. La revoca dell'autorizzazione è una procedura che può essere applicabile solo dalla Government Authority, nel caso in cui un elettore non sia piu autorizzato al voto. In questo caso, la Government Authority non deve rilasciare l'autorizzazione al voto o se è stata già rilasciata, deve revocarla. Il rischio residuo principale è duplice: da un lato, una revoca non propagata in tempo potrebbe consentire l'uso di un'autorizzazione non più valida; dall'altro, una Government Authority malevola o compromessa potrebbe abusare della procedura di revoca per impedire a elettori legittimi di esprimere la propria preferenza.

8 Threat Models

In questa sezione verranno identificate le entità atte a compromettere il corretto funzionamento del sistema. Per ogni Threat Model, verranno elencate le sue finalità, le metodologie con cui potrà attaccare l'infrastruttura e le capacità/risorse computazionali con cui svolgerà le operazioni.

8.1 Elettore disonesto

Il modello è identificato in un elettore che, seppur seguendo il protocollo di voto scelto, cerca di trarre vantaggio da criticità scoperte nel sistema di voto, ad esempio volendo esercitare il suo voto più di una volta, modificare la scelta fatta o ripudiarla.

Finalità

- **Violazione dell'Unicità**: L'elettore disonesto tenta l'espressione di voti multipli per alterare il peso statistico della propria preferenza.
- **Delegittimazione del Processo**: Sfruttare il ripudio sul voto espresso per creare un'apparenza di corruzione nel sistema, inducendo dubbi sulla correttezza del conteggio finale.
- **Collusione o Vendita del Voto**: Dimostrare a terzi la preferenza espressa, in scenari di coercizione o voto di scambio, violando deliberatamente la proprietà di non trasferibilità della prova del voto e aumentando il rischio di coercizione.

Metodologie di Attacco

- **Replay di autorizzazione**: Sfruttamento della procedura di ottenimento delle credenziali di voto per riutilizzare una credenziale già valida, al fine di esercitare il voto più di una volta, violando quindi l'unicità del voto.
- **Uso improprio di identità o credenziali di terzi**: Utilizzare credenziali cedute volontariamente da familiari, conoscenti o altri elettori per controllare più votazioni. In questo caso l'attacco non mira necessariamente a violare la crittografia del sistema, ma a sfruttare la debolezza del contesto remoto e non presidiato, al fine di aumentare la rilevanza statistica della preferenza espressa.
- **Repudiation Attack / Ripudio Fraudolento**: Disconoscimento formale di un voto correttamente inviato e registrato, sostenendo che l'identità sia stata sottratta o che il sistema abbia fallito la registrazione del voto, al fine di forzare il rilascio di una nuova credenziale di voto e di annullare quella inviata.
- **Proof of vote**: Tentativo di avere una prova ufficiale del voto espresso, salvando eventuali ricevute o prove di voto, al fine di convincere terzi del voto espresso e quindi favorire coercizione o corruzione nel contesto di scambio di voti.

Risorse e Capacità

- **Accesso alle risorse di base**: Ha accesso alle proprie credenziali di voto e al proprio dispositivo, come smartphone o computer.
- **Multi-identità**: Disponibilità di ulteriori identità valide al voto che rendono l'attacco indistinguibile, a livello di rete, da un'operazione lecita.
- **Livello di conoscenza**: Comprensione dettagliata dei flussi di autenticazione tra Government Authority e Client.

8.2 Insider all'interno del Voting Server / Avversario interno privilegiato

Si tratta di un avversario attivo operante all'interno della logica del Voting Server. L'entità, che può essere un amministratore di sistema con privilegi elevati, agisce intenzionalmente per manipolare la corretta verificabilità dei voti da parte del Voting Server, alterando quindi il processo di controllo di duplicati e/o invio di voti leciti al Registro Elettorale.

Finalità

- **Alterazione del metodo di verifica dei voti**: Un insider malevolo potrebbe con i suoi permessi di amministrazione accettare un voto che a priori per le regole stabilite non può essere ritenuto valido, oppure rifiutare un voto che dovrebbe invece essere valido, alterando quindi l'autenticità, unicità e integrità del voto ed erodendo la credibilità del sistema elettorale.
- **Accettazione di duplicati**: L'avversario potrebbe permettere al Voting Server di accettare più voti associati alla stessa autorizzazione o credenziale di voto, violando la proprietà di unicità del voto.
- **Invio di schede di voto non valide**: Siccome il Voting Server ha come obiettivo quello di inviare i voti validi al Registro Elettorale, un insider potrebbe disabilitare i controlli su determinate schede non valide per consentire l'invio di queste schede non valide al Registro Elettorale, invalidando la correttezza del voto.
- **Violazione della Segretezza**: Tentare di de-anonimizzare gli elettori associando l'identità reale alla preferenza espressa, spesso condividendo o vendendo tali dati a terzi.

Metodologie di Attacco

- **Data Correlation**: Analisi dei log di sistema e dei pacchetti ricevuti per correlare i timestamp e gli indirizzi IP dei votanti con le schede memorizzate nel database.
- **Manomissione del Software**: Modifica del codice sorgente del server di voto per intercettare informazioni sensibili delle schede in chiaro, se non adeguatamente cifrate end-to-end, rifiutare voti validi o validare voti illeciti.
- **Omissione Selettiva**: Cancellazione mirata di schede appartenenti a determinati gruppi di elettori.

Risorse e Capacità

- **Accesso Privilegiato**: Pieni permessi di amministrazione del server di voto, avendo accesso alla logica di validazione e ai record del server.
- **Capacità Operativa**: Potere di manipolare direttamente i record del database e i log del server.
- **Potenza Computazionale**: Disponibilità delle elevate risorse di calcolo e di storage del Voting Server per eseguire analisi di massa sui dati cifrati.

Si assume che l'avversario non sia in grado di rompere le primitive crittografiche correttamente implementate. La forza dell'avversario deriva principalmente dall'accesso al Voting Server e dalla possibilità di alterarne la logica interna di validazione.

8.3 Registro Elettorale compromesso o disonesto

Si considera un avversario interno o una compromissione della componente incaricata di conservare e pubblicare i voti validamente registrati. Nel modello, il Registro Elettorale supporta la verificabilità individuale e universale; pertanto, una sua compromissione può compromettere la fiducia nel risultato finale anche senza violare direttamente la segretezza del voto.

Finalità

- **Alterazione dell'integrità del registro**: modificare, rimuovere o sostituire schede già registrate.
- **Omissione selettiva**: non pubblicare alcune schede validamente accettate dal Voting Server.
- **Compromissione della verificabilità**: impedire agli elettori o agli osservatori esterni di controllare correttamente l'inclusione dei voti e la coerenza del risultato finale.

Metodologie di Attacco

- **Cancellazione o sostituzione di schede**: tentativo di rimuovere o modificare voti precedentemente registrati.
- **Pubblicazione incompleta**: pubblicazione di una vista parziale del registro, omettendo selettivamente alcune schede.
- **Ritardo nella pubblicazione**: ritardare la disponibilità dei dati necessari alla verifica individuale o universale.
- **Fork del registro**: presentare versioni differenti del registro a osservatori diversi, generando incoerenza nella verifica.

Risorse e Capacità

- **Accesso ai dati registrati**: possibilità di leggere e tentare di modificare le schede presenti nel registro.
- **Controllo sulla pubblicazione**: capacità di decidere quali informazioni rendere visibili e quando.
- **Capacità operativa**: possibilità di interferire con la disponibilità del registro o con le prove di inclusione.

Si assume che il Registro Elettorale compromesso non sia in grado di rompere primitive crittografiche correttamente implementate. La sua forza deriva dal controllo sulla conservazione e pubblicazione dei dati, non dalla capacità di violare direttamente cifratura o firme.

8.4 Voting Client compromesso o disonesto

L'avversario è configurato come un'entità esterna al modello che tenta di distribuire un software malevolo di voto all'elettore onesto simulando fedelmente l'applicazione ufficiale, compromettendo la fase iniziale di espressione della preferenza, o che cerca di manipolare l'applicativo ufficiale. Essendo il client il mezzo con cui l'elettore interagisce per la compilazione della scheda di voto, un eventuale software compromesso potrebbe osservare la preferenza selezionata prima che essa venga trasmessa al sistema.

Finalità

- **Impersonificazione e Voto Fraudolento**: Il software riesce ad ottenere le informazioni di autenticazione per esprimere un voto al posto dell'elettore legittimo, violando la proprietà di Autenticità.
- **Alterazione della preferenza di voto**: Il client malevolo potrebbe mostrare all'elettore onesto la preferenza selezionata dall'elettore, ma inviare successivamente al Voting Server una preferenza diversa e/o annullarne la trasmissione al Voting Server.
- **Violazione dello pseudonimato**: Il client potrebbe cercare di collegare autenticazione dell'utente reale e preferenza espressa, violando quindi lo pseudonimato.
- **Facilitazione della Coercizione**: Il client malevolo potrebbe salvare i dati della scheda elettorale e della preferenza di voto, inducendo l'elettore a modificare la sua scelta per conto di entità terze, favorendo coercizione.
- **Manipolazione della Verificabilità Individuale**: Impedire all'elettore di accorgersi che il proprio voto è stato alterato o soppresso, fornendo una ricevuta visiva falsa che non corrisponde a quanto effettivamente inviato al server.

Metodologie di Attacco

- **Phishing**: Diffusione di link o eseguibili tramite canali non ufficiali, come email, SMS o social, per indurre l'utente a utilizzare l'interfaccia malevola. In questo

modo, quando l'elettore onesto utilizzerà l'applicativo non ufficiale, fornirà i suoi dati sensibili al software malevolo.

- **Conservazione delle informazioni sensibili**: Il client memorizza localmente autorizzazioni, identificativi, ricevute e preferenze per ricostruire il legame tra elettore e voto.
- **Fake Interface**: Creazione di un portale web o di un'applicazione speculare a quella legittima per ingannare l'utente nella fase di inserimento delle credenziali.
- **Credential/Token Forwarding**: Una volta ottenute le credenziali o l'autorizzazione di voto, il client malevolo inoltra queste informazioni alle autorità reali in tempo reale per ottenere i certificati di voto, rendendo l'attacco trasparente all'utente.

Risorse e Capacità

- **Controllo del dispositivo**: L'avversario può controllare totalmente o parzialmente l'ambiente in cui viene eseguito il Voting Client, ad esempio iniettando malware o tramite il client modificato.
- **Ingegneria Sociale e Web Hosting**: Capacità di orchestrare campagne di comunicazione efficaci per l'invio di link malevoli e gestire infrastrutture di hosting per domini malevoli, come cybersquatting o typosquatting.
- **Manipolazione Front-end**: Competenza avanzata nella manipolazione dell'interfaccia utente e dell'esperienza utente per rendere indistinguibile il client falso da quello autentico.
- **Capacità computazionale elevata**: Possesso di uno o più server in grado di eseguire script o algoritmi complessi in poco tempo.
- **Analisi dei Protocolli**: Conoscenza del flusso di comunicazione tra Client, Government Authority e Server per intercettare e reinviare correttamente i messaggi di autorizzazione.
- **Accesso a dati temporanei**: L'avversario può accedere a autorizzazioni, messaggi, conferme o altri dati gestiti localmente durante il protocollo.

8.5 Attaccante esterno

Si definisce come un'entità esterna che non possiede privilegi di accesso diretto alle infrastrutture fisiche o logiche del modello, ma opera attivamente o passivamente sui canali di comunicazione che collegano gli attori del sistema, cioè Elettore, Government Authority e Voting Server.

Può essere definita come:

- **Attaccante passivo**: Un'entità che monitora i canali di comunicazione senza alterare il flusso dei dati o interagire direttamente con i componenti del sistema.
- **Attaccante attivo**: Un avversario che interviene direttamente sulla componente comunicativa, alterando, iniettando, ritardando o eliminando i messaggi scambiati tra le parti oneste, deviando attivamente dal protocollo.

Finalità

- **Compromissione dello pseudoanonimato**: Correlare l'identità reale dell'elettore alla preferenza espressa tramite l'analisi dei flussi comunicativi tra gli attori del sistema.
- **Violazione dell'Integrità e dell'Unicità**: Tentare di alterare il conteggio o inserire voti multipli fraudolenti.
- **Attacco alla Disponibilità e alla Legittimità**: Impedire il corretto svolgimento delle operazioni di voto per determinate fasce di elettori o per l'intera infrastruttura, inducendo gli elettori alla contestazione o al ripudio del voto.

Metodologie di Attacco

- **Replay Attack**: Intercettazione delle credenziali degli utenti o del contenuto dei payload, con l'intento di utilizzarli in contesti fraudolenti per violare il vincolo del voto unico.
- **Attacchi Denial-of-Service (DoS)**: Rendere indisponibili le parti oneste del sistema, ad esempio sovraccaricando i server.
- **Man-in-the-Middle (MitM)**: Intercettazione di credenziali o token di autenticazione durante la comunicazione tra il votante lecito e le entità oneste del sistema; l'attaccante può bloccare, effettuando un attacco alla disponibilità, o alterare il contenuto del payload per minare alla legittimità.
- **Eavesdropping e Analisi del Traffico**: Monitoraggio passivo del traffico tra l'utente e le autorità per estrarre dati atti a identificare pattern comportamentali e dati sensibili violando così la segretezza.
- **Sabotaggio della Fiducia**: Sfruttamento di bug minori, latenze o inefficienza di sistema per amplificare dubbi sulla sicurezza, spingendo gli utenti a dichiarare falsamente che il proprio voto è stato manipolato e spingendo al ripudio del voto a quest'ultimi.

Risorse e Capacità

- **Capacità Computazionali**: Esse possono variare dipendentemente dall'organizzazione che genera questo tipo di attacchi. L'attaccante può avere accesso a più dispositivi per eseguire analisi massive di dati in transito su canali non protetti o eseguire attacchi DoS compromettendo il funzionamento dei server.
- **Capacità di intercettazione**: Capacità di intercettazione su canali non protetti.

8.6 Government Authority Disonesta

L'avversario si configura come l'opposto della parte onesta rappresentata dalla Government Authority, cioè l'entità incaricata di identificare l'elettore, verificarne l'idoneità e rilasciare un'autorizzazione valida al voto. Nel modello aggiornato, questa entità occupa una posizione particolarmente critica perché conosce l'identità reale dell'elettore, il suo stato

di idoneità e lo stato delle autorizzazioni rilasciate. Configurandosi come disonesta, può alterare il processo di identificazione o autorizzazione, rilasciando credenziali indebite, negandole ad elettori legittimi o conservando informazioni utili per correlare identità e successive fasi di partecipazione al voto.

Finalità

- **Autenticazione fraudolenta**: può considerare autenticato un soggetto non legittimo oppure negare l'autenticazione a un elettore legittimo.
- **Rilascio di autorizzazioni indebite**: Può rilasciare credenziali o autorizzazioni al voto a soggetti non aventi diritto, consentendo la partecipazione illegittima all'elezione.
- **Negazione di autorizzazioni valide**: Revoca dell'autorizzazione al voto per specifici elettori legittimi, impedendo loro di esercitare il diritto di voto e compromettendo quindi la correttezza del processo elettorale.
- **Conoscenza dell'identità degli elettori**: accesso ai dati identificativi degli elettori durante la fase di identificazione e verifica dell'idoneità.
- **Collegamento identità-voto**: Avendo accesso diretto all'identità di ogni elettore per poterne verificare l'idoneità al voto, potrebbe conservare informazioni dettagliate su ogni elettore, favorendo quindi la creazione di un collegamento tra l'identità reale dell'elettore e la successiva operazione di voto.

Metodologie di Attacco

- **Manipolazione del registro aventi diritto**: Marcando ad esempio come non idoneo un elettore legittimo o come idoneo un soggetto non autorizzato.
- **Emissione fraudolenta di autorizzazioni**: Rilascio intenzionale di autorizzazioni o credenziali di voto non giustificate da una corretta autenticazione e verifica di idoneità.
- **Emissione multipla per lo stesso elettore**: Rilascio di più autorizzazioni per una stessa identità reale, violando la proprietà di unicità.
- **Collusione con altre componenti**: Condivisione intenzionale di dati con altre entità, al fine di facilitare la correlazione tra identità e voto registrato.

Risorse e Capacità

- **Accesso al registro degli aventi diritto al voto**: Possibilità di modificare e leggere lo stato degli elettori ammessi alla votazione.
- **Controllo fase di autorizzazione**: Capacità di revocare, negare, rilasciare o duplicare autorizzazioni di voto.
- **Conoscenza dell'identità degli elettori**: Accesso ai dati identificativi degli elettori autenticati tramite Identity Service Provider.
- **Accesso a log di autorizzazione**: Disponibilità di informazioni temporali relative alle richieste di autorizzazione.

È doveroso inoltre precisare che non si assume che la Government Authority malevola sia in grado di rompere primitive crittografiche implementate correttamente. La sua forza deriva unicamente dal ruolo istituzionale e operativo che ricopre, potendo controllare la decisione su chi può ricevere diritto al voto e conoscere l'identità degli elettori.

8.7 Scrutiny Authority compromessa o disonesta

Si definisce come un avversario interno che corrompe il corretto funzionamento dell'autorità di scrutinio oppure come una Scrutiny Authority stessa malevola. Tale entità gode di privilegi elevati nella fase finale dell'elezione, poiché il suo compito onesto è calcolare l'esito finale a partire dai voti presenti nel Registro Elettorale e pubblicare un risultato verificabile.

Nel modello proposto, la Scrutiny Authority non dovrebbe conoscere l'identità reale degli elettori, né dovrebbe modificare l'insieme dei voti registrati. Tuttavia, se compromessa o disonesta, può tentare di alterare la correttezza dello scrutinio, pubblicare un risultato non coerente con i voti registrati oppure collaborare con altre componenti per indebolire la segretezza del voto.

Finalità

- **Falsificazione del conteggio**: Pubblicare un risultato finale diverso da quello ottenibile applicando correttamente le regole di scrutinio ai voti presenti nel Registro Elettorale.
- **Omissione selettiva di voti registrati**: Escludere dal conteggio voti regolarmente presenti nel Registro Elettorale, con l'obiettivo di alterare l'esito finale.
- **Delegittimazione del processo**: Pubblicare risultati incompleti, incoerenti o non verificabili, inducendo dubbi sulla correttezza della procedura elettorale.
- **Violazione della segretezza**: Tentare di ricavare informazioni sulle preferenze individuali durante la fase di scrutinio, soprattutto in caso di accesso a dati ausiliari o collusione con altre componenti del sistema.
- **Collusione per la de-anonimizzazione**: Collaborare con Government Authority, Voting Server o Registro Elettorale per tentare di collegare identità reale, autorizzazione e scheda registrata.

Metodologie di Attacco

- **Conteggio scorretto**: Applicazione intenzionalmente errata delle regole di scrutinio previste per il referendum binario.
- **Esclusione selettiva**: Mancato conteggio di alcune schede presenti nel Registro Elettorale.
- **Pubblicazione fraudolenta del risultato**: Pubblicazione di un risultato finale non coerente con l'insieme dei voti registrati.
- **Falsificazione o omissione delle informazioni di verifica**: Mancata pubblicazione delle informazioni necessarie a consentire la verifica universale del risultato.

- **Collusione con altre entità**: Condivisione di informazioni con altre componenti del sistema per tentare la correlazione tra elettori e schede.

Risorse e Capacità

- **Accesso ai dati di scrutinio**: Disponibilità dei voti presenti nel Registro Elettorale necessari per il calcolo del risultato finale.
- **Controllo della procedura di conteggio**: Capacità di influenzare o manipolare il calcolo dell'esito elettorale.
- **Controllo sulla pubblicazione del risultato**: Capacità di pubblicare risultati finali, risultati parziali o informazioni di verifica non coerenti con i voti registrati.
- **Capacità di collusione**: Possibilità di condividere dati o metadati con altre componenti del sistema, al fine di compromettere pseudoanonimato, segretezza o trasparenza.

Si assume che la Scrutiny Authority compromessa non possa modificare direttamente l'elenco degli aventi diritto o rilasciare autorizzazioni di voto, poiché tali funzioni appartengono alla Government Authority. Inoltre, non si assume che possa rompere primitive crittografiche correttamente implementate. La sua forza deriva dal controllo sulla fase di conteggio e dalla possibilità di pubblicare un risultato finale scorretto o non sufficientemente verificabile.

9 Priorità, Compromessi e Rischi Residui

Il modello proposto prioritizza autenticità, unicità, integrità e verificabilità individuale, poiché tali proprietà sono necessarie per garantire che solo elettori legittimi possano partecipare, che ciascuno possa esprimere al più un voto valido e che il risultato finale sia controllabile dagli elettori e da osservatori esterni.

La segretezza del voto e lo pseudoanonimato vengono perseguiti tramite la separazione tra autenticazione e autorizzazione, raccolta del voto e scrutinio. Tuttavia, il modello non garantisce anonimato assoluto: una collusione tra Government Authority, Voting Server, Registro Elettorale e Scrutiny Authority potrebbe consentire la correlazione tra identità reale, autorizzazione e scheda registrata.

La verificabilità individuale richiede che l'elettore possa controllare l'inclusione della propria scheda nel Registro Elettorale. Tuttavia, tale verifica non deve trasformarsi in una prova pubblicamente utilizzabile della preferenza espressa, altrimenti aumenterebbe il rischio di coercizione e compravendita del voto.

Il modello non elimina completamente coercizione e voto di scambio, poiché il voto avviene in un ambiente remoto non presidiato. Un elettore potrebbe essere controllato fisicamente da terzi, utilizzare un client compromesso oppure produrre prove esterne della propria scelta, ad esempio tramite registrazioni o screenshot. Il sistema mira quindi a ridurre la produzione di prove trasferibili, ma non può garantire la stessa protezione offerta da un seggio fisico controllato.

L'efficienza viene trattata come vincolo progettuale. Il modello deve evitare interazioni troppo lunghe, concentrazione eccessiva del carico su una singola componente e meccanismi non proporzionati al contesto applicativo. La valutazione quantitativa di tempi, dimensione dei messaggi e costi computazionali sarà demandata al WP4.

Algoritmi e Protocolli per la Sicurezza

Project Work

WP2: Soluzione proposta

E-Vote: Protocollo per il voto sicuro

Gruppo 16: Gennaro Foschillo, Francesco Vittorio Allocca

A.A. 2025–2026

Indice

1 WP2: Soluzione proposta **1**

1.1 Architettura generale . . . 1

1.2 Glossario e notazione . . . 1

1.2.1 Entità principali . . . 1

1.2.2 Chiavi e certificati . . . 2

1.2.3 Oggetti elettorali e registri . . . 3

1.2.4 Token di autorizzazione . . . 4

1.2.5 Primitive e operatori . . . 4

1.2.6 Scrutiny Authority distribuita . . . 5

1.2.7 Convenzioni sui messaggi . . . 5

1.2.8 Distinzioni terminologiche . . . 6

1.3 Obiettivo del Work Package . . . 6

1.4 Entità del protocollo e informazioni gestite . . . 6

1.4.1 Government Authority (GA) . . . 7

1.4.2 Voting Client (VC) . . . 8

1.4.3 Voting Server (VS) . . . 9

1.4.4 Registro Elettorale . . . 10

1.4.5 Scrutiny Authority (SA) . . . 11

1.5 Primitive crittografiche adottate . . . 12

1.6 Setup iniziale e infrastruttura di fiducia . . . 14

1.6.1 Generazione delle chiavi infrastrutturali . . . 14

1.6.2 Preparazione della Scrutiny Authority distribuita . . . 15

1.6.3 Richiesta di accreditamento . . . 16

1.6.4 Verifica della richiesta da parte della Government Authority . . . 17

1.6.5 Emissione dei certificati . . . 17

1.6.6 Pubblicazione del manifesto dell'elezione . . . 18

1.6.7 Pubblicazione e aggiornamento della lista di revoca . . . 19

1.6.8 Revoca dell'accreditamento . . . 20

1.6.9 Premessa sui canali di comunicazione sicuri . . . 21

1.7 Flusso generale del protocollo . . . 21

1.8 Fase 1: inizializzazione del Voting Client 22
1.9 Fase 2: autenticazione dell'elettore 23
1.10 Fase 3: verifica dell'idoneità e rilascio del token 24
1.11 Fase 4: preparazione locale della scheda 24
1.12 Fase 5: verifica del Voting Server e apertura del canale sicuro 26
1.13 Fase 6: invio della busta di voto e verifica del token 27
1.14 Fase 7: registrazione del token speso 28
1.15 Fase 8: pubblicazione sul Registro Elettorale 29
1.16 Fase 9: chiusura delle urne e scrutinio finale 31
1.17 Fase 10: pubblicazione del risultato e verificabilità 33
1.17.1 Verificabilità individuale 34

1 WP2: Soluzione proposta

1.1 Architettura generale

La figura illustra l'architettura generale del sistema, evidenziando le principali entità coinvolte (utente, Voting Client, Voting Server, Registro Elettorale, Scrutiny Authority, nodi della Scrutiny Authority e Government Authority) e i flussi di comunicazione tra esse.

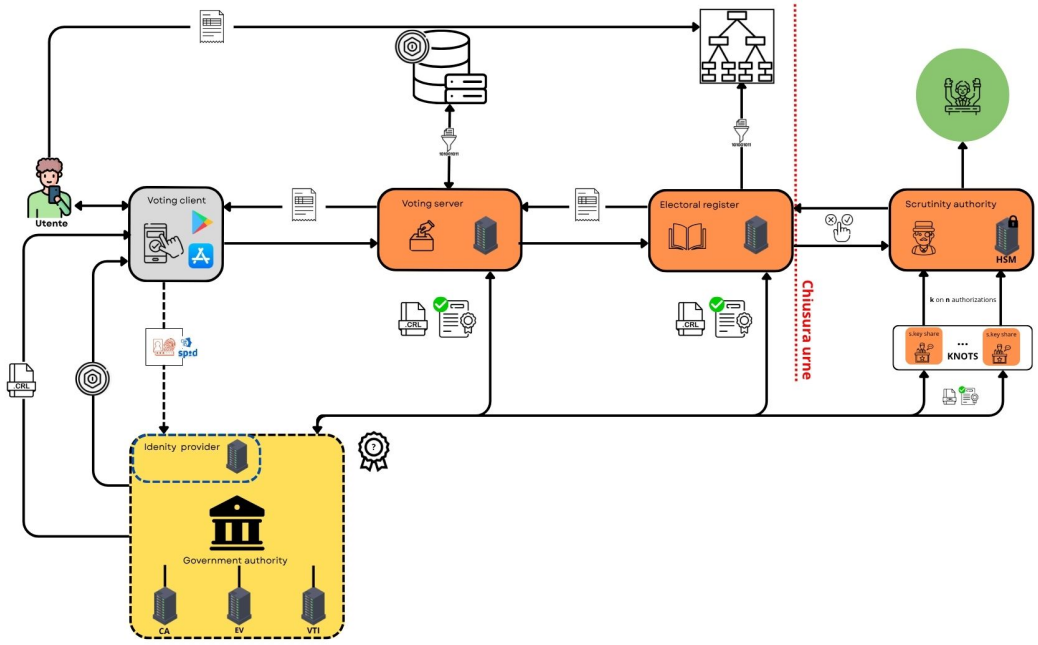


Figura 1: Architettura del protocollo.

1.2 Glossario e notazione

Questa sezione introduce la notazione usata nel protocollo. Le definizioni sono volutamente sintetiche in questa parte, in quanto verranno poi specificate nelle sezioni successive. Nel seguito si considera una specifica istanza elettorale identificata da $ID_{Election}$.

1.2.1 Entità principali

Simbolo	Descrizione
U	Elettore che partecipa alla consultazione.
VC	Voting Client usato dall'elettore per autenticarsi, ottenere il token, compilare la scheda e inviare il voto.
GA	Government Authority responsabile di autenticazione, verifica di idoneità, rilascio dei token, certificazione delle entità e pubblicazione dei parametri elettorali.

Simbolo	Descrizione
IP	Identity Provider Module della GA, incaricato di gestire logicamente l'autenticazione dell'elettore appoggiandosi a un'infrastruttura di identità digitale esterna riconosciuta, ad esempio SPID, CIE o servizio equivalente.
CA	Certification Authority Module della GA, incaricato di emettere e revocare i certificati delle entità infrastrutturali.
EV	Eligibility Verification Module della GA, incaricato di verificare il diritto al voto dell'elettore autenticato.
VTI	Voting Token Issuer Module della GA, incaricato di emettere token di autorizzazione al voto.
VS	Voting Server che verifica i token ricevuti, accetta le schede cifrate valide e le inoltra al Registro Elettorale.
ER	Registro Elettorale / Bulletin Board append-only che conserva e pubblica le schede cifrate accettate.
SA	Scrutiny Authority incaricata dello scrutinio finale. Nel protocollo è modellata come autorità distribuita.
SA_i	Nodo fiduciario della Scrutiny Authority distribuita, con $i \in \{1, \ldots, n\}$.

1.2.2 Chiavi e certificati

Con pk_X e sk_X si indicano rispettivamente la chiave pubblica e la chiave privata dell'entità X.

Simbolo	Descrizione
(pk_{GA}, sk_{GA})	Coppia di chiavi della Government Authority. sk_{GA} firma certificati, token, manifesto e CRL; pk_{GA} verifica tali firme.
(pk_{VS}, sk_{VS})	Coppia di chiavi del Voting Server. pk_{VS} è inclusa in $Cert_{VS}$; sk_{VS} autentica il server e firma eventuali messaggi verso ER o VC.
(pk_{ER}, sk_{ER})	Coppia di chiavi del Registro Elettorale. sk_{ER} firma record, stati del registro o ricevute di inclusione.
pk_{SA}	Chiave pubblica elettorale della Scrutiny Authority, generata internamente dall'Hardware Security Module insieme alla corrispondente chiave privata sk_{SA}. La chiave pk_{SA} è esportabile e viene pubblicata in $Params_{Election}$, così che i Voting Client possano usarla per cifrare la preferenza.

Simbolo	Descrizione
sk_{SA}	Chiave privata elettorale associata a pk_{SA}, generata e custodita internamente dall'Hardware Security Module della Scrutiny Authority. Nel protocollo proposto sk_{SA} è marcata come non esportabile: nessuna entità software esterna, inclusi i nodi della Scrutiny Authority, può leggerla direttamente. Le operazioni di decifratura vengono eseguite internamente dall'HSM solo dopo una procedura congiunta autorizzata da almeno k nodi della Scrutiny Authority. Non si assume una threshold decryption matematica.
(pk_{SA_i}, sk_{SA_i})	Coppia di chiavi infrastrutturali del nodo SA_i, certificata dalla Government Authority. sk_{SA_i} serve ad autenticare il nodo e a firmare le autorizzazioni o i messaggi prodotti durante lo scrutinio;
(pk_U, sk_U)	Coppia di chiavi dell'elettore per la specifica elezione. pk_U vincola il token; sk_U resta nel client o nel wallet locale.
$Cert_X$	Certificato digitale dell'entità X, firmato dalla GA. Associa chiave pubblica, identità, ruolo e istanza elettorale.
$Cert_{VS}$	Certificato del Voting Server.
$Cert_{ER}$	Certificato del Registro Elettorale.
$Cert_{SA_i}$	Certificato del nodo fiduciario SA_i.

1.2.3 Oggetti elettorali e registri

Simbolo	Descrizione
$ID_{Election}$	Identificativo univoco dell'elezione.
$Params_{Election}$	Manifesto pubblico dell'elezione: contiene identificativo, opzioni, tempi, certificati, pk_{SA}, parametri della SA distribuita, versione, timestamp e firma della GA.
$CRL_{Election}$	Lista dei certificati revocati per l'elezione, firmata dalla GA.
$EligibleEntitiesDB$	Registro interno della GA con le entità infrastrutturali accreditabili.
$EligibleUsersDB$	Registro interno della GA con gli elettori aventi diritto.
$SpentTokensDB$	Registro interno del VS usato per rilevare il riuso dei token senza salvarli in chiaro.
$IssuedTokensDB$	Registro interno della Government Authority che mantiene lo stato dei token emessi per impedire il rilascio di più token allo stesso elettore nella stessa elezione.
$Options$	Insieme delle opzioni ammesse. $Options = \{\text{Sì}, \text{No}\}$.
v	Preferenza selezionata dall'elettore, con $v \in Options$.
$EncVoto_{SA}$	Voto cifrato con pk_{SA}.

Simbolo	Descrizione
$BallotCommit_U$	Commitment crittografico calcolato dal Voting Client sulla scheda cifrata prodotta localmente. Viene conservato localmente dal client e inserito nella busta di voto, nel record pubblicato e nella ricevuta firmata, per permettere all'elettore di rilevare eventuali sostituzioni della scheda cifrata da parte del Voting Server.
$Ballot$	Busta logica di voto inviata dal Voting Client al Voting Server.
$BoardRecord_i$	Record pubblicato dal Registro Elettorale per la scheda cifrata i-esima.
$Leaf_i$	Hash della scheda cifrata o del relativo record, usato per la verifica di inclusione.
$Root_{final}$	Radice finale della struttura pubblicata dal Registro Elettorale alla chiusura delle urne.
$Receipt_U$	Ricevuta firmata dal Registro Elettorale contenente il riferimento crittografico alla scheda accettata. Non costituisce da sola una prova Merkle completa; la verifica di inclusione viene completata dopo la pubblicazione di $Root_{final}$ e del relativo $MerklePath_i^{final}$.

1.2.4 Token di autorizzazione

Simbolo	Descrizione
$SerialNumber_{Token}$	Identificativo univoco del token generato dalla GA.
$Permit_U$	Permesso di voto: contiene almeno $ID_{Election}$, $SerialNumber_{Token}$, pk_U ed eventuale scadenza. Non contiene la preferenza.
σ_{GA}^{Permit}	Firma della GA sul permesso $Permit_U$.
$VotingToken_U$	Token firmato: $VotingToken_U = (Permit_U, \sigma_{GA}^{Permit})$.
$EncToken_U$	Token cifrato dalla GA verso il Voting Client usando pk_U.
K_{Master_Server}	Segreto interno del VS usato per derivare una chiave deterministica interna.
$K_{Spent}^{Election}$	Chiave interna derivata dal Voting Server per la specifica elezione, calcolata come $H(K_{Master_Server} \parallel ID_{Election})$. Una chiave deterministica usata per calcolare il tag HMAC dei token spesi.
$LookupTag_{Token}$	Tag deterministico di lookup calcolato dal Voting Server come $HMAC_{K_{Spent}^{Election}}(SerialNumber_{Token})$. Viene memorizzato in $SpentTokensDB$ per rilevare il riuso del token senza salvare il token completo o il suo numero seriale in chiaro.

1.2.5 Primitive e operatori

Notazione	Descrizione
$H(m)$	Hash crittografico del messaggio m.
$Sign_{sk_X}(H(m))$	Firma digitale prodotta da X sul digest di m.
$Verify_{pk_X}(m, \sigma)$	Verifica della firma σ tramite la chiave pubblica pk_X.
$Enc_{pk_X}(m)$	Cifratura asimmetrica di m per l'entità X.
$Dec_{sk_X}(c)$	Decifratura del ciphertext c tramite sk_X.
$MAC_k(m)$	Codice di autenticazione del messaggio m tramite chiave simmetrica k.
$\|$	Concatenazione ordinata di campi.
$nonce$	Valore casuale monouso usato per freschezza e anti-replay.
$timestamp$	Istante temporale associato a messaggi, certificati o liste di revoca.
$k_{session}$	Chiave simmetrica di sessione stabilita tramite canale sicuro autenticato.
k_{enc}, k_{mac}	Sotto-chiavi derivate da $k_{session}$ per cifratura e autenticazione del canale.

1.2.6 Scrutiny Authority distribuita

Simbolo	Descrizione
n	Numero totale di nodi fiduciari della Scrutiny Authority.
k	Numero minimo di nodi necessari per autorizzare lo scrutinio finale, con $1 \leq k \leq n$.
$\langle k, n \rangle$	Parametri della procedura distribuita di autorizzazione allo scrutinio.
HSM_{SA}	Hardware Security Module, o infrastruttura equivalente, associato alla Scrutiny Authority. Genera internamente la coppia elettorale (pk_{SA}, sk_{SA}), esporta solo pk_{SA}, custodisce sk_{SA} come chiave non esportabile ed esegue internamente le operazioni di decifratura dopo la verifica di almeno k approvazioni valide.
$Approval_i$	Autorizzazione firmata dal nodo SA_i alla partecipazione allo scrutinio finale.
$TallyBoard$	Bacheca finale pubblicata per rendere verificabile lo scrutinio.

1.2.7 Convenzioni sui messaggi

I messaggi del protocollo sono indicati nella forma:

$$M_j : A \rightarrow B : (campo_1, campo_2, \ldots, campo_n)$$

dove M_j identifica il messaggio, A è il mittente e B è il destinatario.

Una firma viene indicata come:

$$\sigma_X = Sign_{sk_X}(H(m))$$

mentre una cifratura per il destinatario Y viene indicata come:

$$Enc_{pk_Y}(m).$$

1.2.8 Distinzioni terminologiche

Nel documento, *autenticazione* indica la verifica dell'identità di un elettore o di un'entità infrastrutturale. *Autorizzazione* indica invece il rilascio del diritto a votare per una specifica elezione.

La *segretezza del voto* riguarda la protezione della preferenza. Il *pseudoanonimato* riguarda la separazione tra identità reale e scheda nelle fasi di raccolta, registrazione e scrutinio.

La *verificabilità individuale* permette al singolo elettore di controllare l'inclusione della propria scheda cifrata. La *verificabilità universale* permette a osservatori esterni di controllare la coerenza tra schede pubblicate, scrutinio e risultato finale.

1.3 Obiettivo del Work Package

In questo Work Package viene descritta la soluzione proposta a partire dal modello logico definito nel WP1. La soluzione è progettata per un referendum binario, in cui l'elettore può esprimere una preferenza tra le opzioni ammesse dalla consultazione. Questa scelta riduce la complessità della scheda, semplifica la fase di scrutinio e consente di concentrare la progettazione sui meccanismi di autenticazione, autorizzazione, segretezza del voto, integrità della registrazione e verificabilità del risultato.

L'obiettivo principale è definire un protocollo che realizzi un compromesso ragionevole tra autenticità, unicità, segretezza, pseudoanonimato, integrità, verificabilità individuale, verificabilità universale ed efficienza. La soluzione ha come obiettivo quello di separare quanto più possibile le informazioni trattate dalle varie componenti, evitando associazioni dirette identità-voto.

In particolare, il protocollo specifica le azioni sequenziali eseguite dalle parti oneste, le chiavi e i certificati utilizzati dalle entità infrastrutturali, il formato logico dei principali messaggi scambiati, il meccanismo di autenticazione dell'elettore e di rilascio del token di autorizzazione, la preparazione e registrazione della scheda cifrata, la procedura di scrutinio finale e i meccanismi di verificabilità individuale e universale.

Nel protocollo proposto inoltre, non viene supportata la sostituzione e l'annullamento del voto dopo l'invio. La scelta è motivata dalla volontà di mantenere semplice il vincolo di unicità: un token valido può produrre al più una scheda accettata e pubblicata. L'introduzione della sostituzione del voto richiederebbe una gestione più complessa dello stato dei token e delle versioni delle schede, con impatti su pseudoanonimato, verificabilità ed efficienza.

1.4 Entità del protocollo e informazioni gestite

In questa sezione vengono descritte le principali entità coinvolte nel protocollo proposto. Per ciascuna entità vengono specificati il ruolo svolto, le chiavi possedute, le informazioni

gestite e i dati che non devono essere conosciuti per preservare la separazione tra identità reale, autorizzazione al voto e preferenza espressa.

1.4.1 Government Authority (GA)

Descrizione. La Government Authority rappresenta l'autorità istituzionale responsabile della gestione dell'elezione dal punto di vista dell'identificazione degli elettori, dell'autorizzazione al voto e della fiducia infrastrutturale. Nel protocollo proposto è modellata come un'unica entità composta da più moduli logici interni, ciascuno associato a una specifica responsabilità.

Moduli logici. La Government Authority include i seguenti moduli:

- **Identity Provider Module (IP):** gestisce logicamente la fase di identificazione e autenticazione dell'elettore che richiede di partecipare all'elezione. Nel protocollo proposto, il modulo IP non implementa necessariamente in modo autonomo l'intera infrastruttura di identità digitale, ma si appoggia a un'infrastruttura esterna riconosciuta dalla Government Authority, ad esempio SPID, CIE o un servizio equivalente. L'esito dell'autenticazione viene acquisito dalla Government Authority e usato come input per la successiva verifica di idoneità elettorale.
- **Certification Authority Module (CA):** certifica le entità infrastrutturali autorizzate a partecipare al protocollo, cioè Voting Server, Registro Elettorale e nodi fiduciari della Scrutiny Authority. Il modulo rilascia certificati digitali che associano una chiave pubblica a un'identità infrastrutturale, a un ruolo e a una specifica elezione. Inoltre, mantiene le informazioni relative ai certificati emessi e revocati.
- **Eligibility Verification Module (EV):** verifica che un elettore autenticato sia effettivamente avente diritto al voto per l'elezione considerata. Questo modulo utilizza il registro degli aventi diritto e non tratta informazioni relative alla preferenza di voto.
- **Voting Token Issuer Module (VTI):** rilascia token di autorizzazione unici e validi per una specifica elezione che permettono all' elettore di accedere alla compilazione della scheda. I token vengono associati internamente all'identità reale dell'elettore, ma tale associazione rimane confinata nella Government Authority.

La Government Authority è inoltre responsabile della pubblicazione dei parametri pubblici dell'elezione, indicati come $Params_{Election}$, e della lista dei certificati revocati, indicata come $CRL_{Election}$.

Chiavi. La Government Authority possiede una coppia di chiavi di firma (pk_{GA}, sk_{GA}). La chiave privata sk_{GA} viene utilizzata per firmare certificati digitali, token di autorizzazione, parametri pubblici dell'elezione e liste di revoca. La chiave pubblica pk_{GA} viene utilizzata dalle altre entità per verificare gli oggetti firmati dalla Government Authority.

Dati che conosce. La Government Authority conosce l'identità reale dell'elettore durante le fasi di identificazione, autenticazione e verifica dell'idoneità. Riceve inoltre la chiave pubblica pk_U, generata dal Voting Client per la specifica elezione, al solo scopo di vincolare il token di autorizzazione al possesso della corrispondente chiave privata

sk_U. La Government Authority conosce anche le identità delle entità infrastrutturali che richiedono un certificato e le rispettive chiavi pubbliche.

Dati che non deve conoscere. La Government Authority non deve conoscere la preferenza espressa dall'elettore e non deve ricevere la scheda cifrata. La sua interazione con l'elettore termina con il rilascio dell'autorizzazione al voto.

1.4.2 Voting Client (VC)

Descrizione. Il Voting Client è l'applicazione ufficiale attraverso cui l'elettore interagisce con il sistema di voto. Permette all'utente di accedere all'istanza elettorale, avviare la fase di autenticazione presso la Government Authority, ottenere il token di autorizzazione, compilare la scheda e inviare il voto.

Assunzione operativa sul client. Per la soluzione proposta si assume che il Voting Client sia ottenuto da un canale ufficiale della Government Authority, ad esempio portale istituzionale, store verificato o repository ufficiale, e che il pacchetto applicativo sia firmato dalla Government Authority. Prima dell'uso, l'elettore o il sistema operativo possono verificare l'autenticità del pacchetto applicativo tramite la firma della GA. Questa assunzione riduce il rischio di installazione di un client falso, ma non elimina il rischio di compromissione del dispositivo dell'utente.

Ruolo. Il Voting Client svolge le seguenti funzioni principali:

- recupera e verifica $Params_{Election}$ e $CRL_{Election}$;
- verifica i certificati delle entità, cioè $Cert_{VS}$, $Cert_{ER}$ e i certificati dei nodi fiduciari della Scrutiny Authority;
- genera o recupera una coppia di chiavi (pk_U, sk_U) valida per la specifica elezione $ID_{Election}$;
- richiede alla Government Authority il token di autorizzazione;
- verifica localmente il token ricevuto, controllando la firma della Government Authority, l'identificativo dell'elezione, la validità temporale e il legame con pk_U;
- mostra all'elettore le opzioni previste dalla consultazione;
- cifra la preferenza utilizzando la chiave pubblica elettorale della Scrutiny Authority;
- firma la richiesta di voto con sk_U, dimostrando il possesso della chiave privata associata al token;
- verifica il certificato del Voting Server e apre un canale sicuro autenticato;
- invia al Voting Server la scheda cifrata e il token di autorizzazione;
- riceve una prova di inclusione utile alla verificabilità individuale.

Chiavi. Il Voting Client non possiede chiavi permanenti valide per più elezioni. Per ogni elezione genera o recupera una coppia di chiavi (pk_U, sk_U) associata a $ID_{Election}$. La chiave pubblica pk_U viene comunicata alla Government Authority durante la richiesta di autorizzazione ed è inserita nel token firmato. La chiave privata sk_U rimane sul dispositivo

dell'utente, preferibilmente in uno storage locale protetto o wallet, e viene utilizzata per dimostrare il possesso della chiave associata al token.

Il Voting Client utilizza inoltre le chiavi pubbliche delle entità infrastrutturali, estratte dai rispettivi certificati, per verificare firme, validare il Voting Server e cifrare la scheda verso la Scrutiny Authority.

Relazione con il canale sicuro. L'apertura del canale sicuro con il Voting Server non richiede che il client sia pienamente fidato. Essa richiede che il client verifichi correttamente $Cert_{VS}$, controlli la $CRL_{Election}$ e partecipi a un key exchange autenticato, ad esempio basato su Diffie-Hellman autenticato dal certificato del server. Il risultato di questa fase è una chiave di sessione $k_{session}$, usata dallo strato di comunicazione per proteggere i messaggi in transito.

Il canale sicuro garantisce che un avversario esterno non possa sostituirsi al Voting Server, leggere o modificare la busta di voto durante la trasmissione o riutilizzare messaggi di sessioni precedenti. Tuttavia, se il client locale è già malevolo o il dispositivo è compromesso, il canale sicuro non impedisce la manipolazione prima dell'invio. Per questo motivo, la sicurezza del protocollo viene fondata tra distribuzione ufficiale del software, verifiche crittografiche, cifratura della scheda verso la Scrutiny Authority e verificabilità successiva dell'inclusione.

Dati che conosce e gestisce temporaneamente. Durante il protocollo, il Voting Client può trattare $Params_{Election}$, $CRL_{Election}$, i certificati delle entità operative, $ID_{Election}$, il token di autorizzazione ricevuto, la coppia (pk_U, sk_U), la preferenza selezionata prima della cifratura, la scheda cifrata e la ricevuta di inclusione.

Dati che non deve conservare stabilmente. Il Voting Client non deve conservare stabilmente informazioni che permettano di collegare identità reale, token di autorizzazione e preferenza espressa. Dopo il completamento della votazione, non deve mantenere la preferenza in chiaro o log locali contenenti dati utili a ricostruire il legame tra elettore e voto.

Rischio residuo. Un client compromesso può alterare la preferenza prima della cifratura, mostrare all'elettore un'interfaccia ingannevole o conservare dati locali utili alla coercizione. Il protocollo può mitigare questi rischi tramite distribuzione ufficiale del client, verifica della firma del pacchetto applicativo, uso di canali sicuri e verificabilità individuale dell'inclusione, ma non può eliminarli completamente in un contesto di voto remoto non presidiato.

1.4.3 Voting Server (VS)

Descrizione. Il Voting Server, dopo essere stato certificato dalla Government Authority, è incaricato di ricevere le schede cifrate inviate dal Voting Client e di verificarne l'autorizzazione tramite il token di voto. Il Voting Server verifica che il token sia valido, firmato dalla Government Authority, riferito all'elezione corrente e non già utilizzato. È inoltre responsabile dell'inoltro delle schede accettate al Registro Elettorale.

Il Voting Server non autentica direttamente l'identità reale dell'elettore e non decide l'idoneità alla partecipazione. Tali funzioni sono svolte dalla Government Authority. Inoltre, non deve conoscere la preferenza in chiaro, poiché riceve solo schede cifrate per

la Scrutiny Authority.

Ruolo. Il Voting Server svolge le seguenti funzioni principali:

- presenta il proprio certificato digitale $Cert_{VS}$ al Voting Client;
- partecipa alla creazione di un canale sicuro con il Voting Client;
- verifica firma, validità e scadenza del token di autorizzazione;
- verifica che il token sia valido per $ID_{Election}$;
- verifica che il token non sia già stato utilizzato;
- registra un riferimento non invertibile al token nel registro dei token spesi;
- inoltra al Registro Elettorale la scheda cifrata validata;
- riceve dal Registro Elettorale una ricevuta di inclusione e la inoltra al Voting Client.

Chiavi. Il Voting Server possiede una coppia di chiavi (pk_{VS}, sk_{VS}). La chiave pubblica pk_{VS} è contenuta nel certificato digitale $Cert_{VS}$, rilasciato e firmato dalla Government Authority. La chiave privata sk_{VS} viene utilizzata per autenticarsi durante l'apertura del canale sicuro e, ove previsto, per firmare messaggi destinati al Registro Elettorale o ricevute inoltrate al Voting Client.

Dati che conosce e gestisce. Durante il protocollo, il Voting Server può conoscere il proprio certificato, $Params_{Election}$, $CRL_{Election}$, i token di autorizzazione ricevuti, lo stato dei token già utilizzati, le schede cifrate ricevute, metadati tecnici di sessione e riferimenti ai record pubblicati sul Registro Elettorale.

Per ridurre l'esposizione dei token in caso di compromissione del database, il Voting Server non conserva il token in chiaro nel registro dei token spesi. Conserva invece un valore derivato mediante hash, sufficiente a riconoscere deterministicamente eventuali riutilizzi dello stesso token.

Dati che non deve conoscere. Il Voting Server non deve conoscere l'identità reale dell'elettore e non deve conoscere la preferenza in chiaro. Può verificare che un token valido sia stato utilizzato, ma non deve poter ricondurre tale token all'identità reale dell'elettore.

1.4.4 Registro Elettorale

Descrizione. Il Registro Elettorale, indicato anche come Bulletin Board, è l'entità incaricata di conservare e pubblicare le schede cifrate accettate dal Voting Server. Nel protocollo proposto viene modellato come un registro append-only, cioè una struttura in cui è possibile aggiungere record, ma non modificarli o rimuoverli senza che l'alterazione risulti rilevabile.

Il Registro Elettorale non autentica gli elettori, non verifica l'idoneità al voto e non decifra le schede ricevute. La sua funzione è rendere pubblicamente verificabile l'insieme dei voti accettati, supportando sia la verificabilità individuale sia la verificabilità universale.

Ruolo. Il Registro Elettorale svolge le seguenti funzioni principali:

- riceve dal Voting Server le schede cifrate che hanno superato i controlli;
- registra ogni scheda accettata in un record append-only;
- calcola le impronte crittografiche dei record;
- collega i record mediante struttura Merkle-based e una Bulletin-Board append-only;
- produce una ricevuta o prova di inclusione;
- rende pubblici i dati necessari alla verifica individuale e universale;
- fornisce, a chiusura dell'elezione, l'insieme dei voti cifrati alla Scrutiny Authority.

Chiavi. Il Registro Elettorale possiede una coppia di chiavi (pk_{ER}, sk_{ER}). La chiave pubblica pk_{ER} è contenuta nel certificato digitale $Cert_{ER}$, rilasciato e firmato dalla Government Authority. La chiave privata sk_{ER} può essere utilizzata per firmare record pubblicati, ricevute di inclusione o stati del registro. Il certificato $Cert_{ER}$, invece, è firmato dalla Government Authority e non dal Registro Elettorale stesso.

Dati che conosce e gestisce. Durante il protocollo, il Registro Elettorale può conoscere il proprio certificato, $Params_{Election}$, $CRL_{Election}$, le schede cifrate accettate dal Voting Server, gli identificativi dei record pubblicati, i timestamp di registrazione, gli hash dei record, le ricevute o prove di inclusione ed eventuali firme del Voting Server.

Dati che non deve conoscere. Il Registro Elettorale non deve conoscere l'identità reale dell'elettore, il token associato all'identità reale e la preferenza in chiaro. Esso conserva solo informazioni pubblicabili relative alle schede cifrate e alla loro inclusione nel registro.

1.4.5 Scrutiny Authority (SA)

Descrizione. La Scrutiny Authority è l'entità incaricata di eseguire lo scrutinio finale dell'elezione. Interviene solo dopo la chiusura delle urne e opera sui voti cifrati pubblicati nel Registro Elettorale. Non partecipa alla fase di identificazione, non verifica l'idoneità, non rilascia autorizzazioni al voto e non riceve le schede direttamente dal Voting Client.

Nel protocollo proposto, la Scrutiny Authority è modellata come un insieme di nodi fiduciari $SA_1, \ldots, SA_n$. La scelta di una Scrutiny Authority distribuita evita che una singola componente possa controllare autonomamente la fase di scrutinio o usare da sola la chiave privata di scrutinio. Lo scrutinio richiede quindi la cooperazione di almeno k nodi su n, secondo una soglia $\langle k, n \rangle$ definita nei parametri pubblici dell'elezione.

Ruolo. La Scrutiny Authority svolge le seguenti funzioni principali:

- rende disponibile la chiave pubblica elettorale pk_{SA}, usata dal Voting Client per cifrare la preferenza;
- protegge l'uso della chiave privata di scrutinio tramite una procedura di custodia e autorizzazione distribuita;
- dopo la chiusura delle urne, recupera dal Registro Elettorale le schede cifrate;
- verifica che i voti da scrutinare corrispondano ai record presenti nel Registro Elettorale;
- autorizza in modo congiunto la procedura di decifratura finale, richiedendo la cooperazione di almeno k nodi fiduciari;

- decifra le schede cifrate tramite la chiave privata di scrutinio sk_{SA}, secondo lo schema di cifratura adottato;
- calcola il risultato finale e pubblica le informazioni necessarie alla verifica.

Chiavi. La Scrutiny Authority utilizza una coppia di chiavi elettorale (pk_{SA}, sk_{SA}). A differenza delle chiavi infrastrutturali dei singoli nodi SA_i, questa coppia non viene generata da un nodo ordinario. La coppia (pk_{SA}, sk_{SA}) viene generata internamente dall'Hardware Security Module associato alla Scrutiny Authority. La chiave pubblica pk_{SA} viene esportata dall'HSM e pubblicata nei parametri dell'elezione $Params_{Election}$, così che i Voting Client possano usarla per cifrare la preferenza. La chiave privata sk_{SA}, invece, resta non esportabile all'interno dell'HSM e non viene mai rivelata in chiaro ai nodi della Scrutiny Authority o ad altre componenti software. Nel protocollo proposto non si assume una costruzione di threshold decryption matematica. Di conseguenza, i nodi SA_i non possiedono quote crittografiche di sk_{SA} e non producono decifrature parziali. Essi partecipano invece alla procedura di autorizzazione dello scrutinio firmando approvazioni $Approval_i$. Quando almeno k approvazioni valide sono disponibili, l'HSM abilita l'uso interno di sk_{SA} per decifrare le schede cifrate. Ciascun nodo fiduciario SA_i possiede inoltre una propria coppia di chiavi infrastrutturali (pk_{SA_i}, sk_{SA_i}), certificata dalla Government Authority. Tali chiavi servono ad autenticare il nodo e a firmare le autorizzazioni prodotte durante la fase di scrutinio. **Dati che conosce e gestisce.** Durante il protocollo, la Scrutiny Authority può conoscere i propri certificati, $Params_{Election}$, $CRL_{Election}$, l'insieme delle schede cifrate presenti nel Registro Elettorale, i record pubblici necessari alla verifica, le autorizzazioni firmate dai nodi fiduciari, le preferenze decifrate durante lo scrutinio e il risultato aggregato finale.

Dati che non deve conoscere. La Scrutiny Authority non deve conoscere l'identità reale degli elettori, i token di autorizzazione associati alle identità reali e le informazioni usate dalla Government Authority per verificare l'idoneità al voto. Essa può partecipare alla decifratura delle schede ai fini del conteggio, ma non deve essere in grado di associare una preferenza decifrata a uno specifico elettore.

1.5 Primitive crittografiche adottate

Questa sezione riassume le primitive crittografiche utilizzate nel protocollo e il loro ruolo operativo. L'obiettivo non è descrivere la teoria delle primitive, ma chiarire in quali punti della soluzione esse vengono applicate e quali proprietà supportano.

Primitiva	**Uso nel protocollo**	**Proprietà supportate**
Firma digitale	La Government Authority firma certificati, token di autorizzazione, manifesto dell'elezione e liste di revoca. Il Registro Elettorale può firmare record, ricevute o stati del registro. I nodi della Scrutiny Authority firmano le autorizzazioni alla procedura di scrutinio finale.	Autenticità, integrità, non ripudio operativo degli oggetti pubblicati.

Primitiva	**Uso nel protocollo**	**Proprietà supportate**
Certificati digitali e PKI	I certificati emessi dalla Government Authority associano una chiave pubblica a un'identità infrastrutturale, a un ruolo e a una specifica elezione. Sono usati per validare Voting Server, Registro Elettorale e nodi fiduciari della Scrutiny Authority.	Distribuzione autenticata delle chiavi pubbliche, mitigazione di sostituzione di chiave e impersonificazione.
Cifratura asimmetrica	Il Voting Client cifra la preferenza usando la chiave pubblica elettorale pk_{SA}. In questo modo Voting Server e Registro Elettorale trattano solo schede cifrate. La cifratura può inoltre essere usata per proteggere il token quando viene consegnato al destinatario previsto.	Segretezza della preferenza, separazione tra raccolta e scrutinio.
Funzioni hash	Gli hash sono usati per calcolare impronte di schede cifrate, record del Bulletin Board, radici Merkle o hash pointer. Sono inoltre usati dal Voting Server per registrare riferimenti non invertibili ai token spesi.	Integrità, rilevazione di alterazioni, supporto alla verificabilità individuale e universale.
MAC e autenticazione del canale	Nei canali sicuri di comunicazione, i MAC o meccanismi equivalenti di autenticazione dei messaggi sono usati per rilevare modifiche ai messaggi in transito. La sola cifratura del canale non viene considerata sufficiente a garantire integrità.	Integrità e autenticazione dei messaggi di sessione.
Nonce e timestamp	Nonce e timestamp sono utilizzati nei messaggi di autenticazione, richiesta token e sessione per legare le risposte al contesto corrente e ridurre il rischio di replay.	Freschezza, protezione da riuso e replay.
Chiavi di sessione	Quando due entità comunicano attraverso un canale sicuro, viene stabilita una chiave simmetrica di sessione, indicata come $k_{session}$. Da essa possono essere derivate sotto-chiavi per cifratura e autenticazione del canale.	Confidenzialità e integrità efficiente delle comunicazioni.

Primitiva	Uso nel protocollo	Proprietà supportate
Custodia distribuita della chiave di scrutinio	La chiave privata di scrutinio associata a pk_{SA} non viene resa utilizzabile da una singola entità durante l'elezione. Lo scrutinio finale richiede una procedura congiunta autorizzata da almeno k nodi su n, i quali firmano la propria approvazione alla fase di decifratura. Nel protocollo base questa scelta non introduce una primitive di threshold decryption, ma riduce il singolo punto di fiducia nella fase di scrutinio.	Riduzione del singolo punto di fiducia, controllo congiunto dello scrutinio, tracciabilità delle autorizzazioni.

Le primitive vengono quindi usate in modo complementare: la cifratura protegge la segretezza della preferenza, le firme digitali e i certificati supportano autenticità e distribuzione fidata delle chiavi pubbliche, gli hash rendono verificabili i dati pubblicati, mentre i meccanismi di autenticazione del canale proteggono i messaggi in transito. Per ragioni di efficienza, il protocollo usa la crittografia asimmetrica solo per autenticare le entità, proteggere oggetti piccoli o stabilire una chiave di sessione. I messaggi successivi viaggiano invece sul canale sicuro basato su $k_{session}$, evitando di applicare operazioni asimmetriche ridondanti a ogni scambio per aumentare l'efficienza del protocollo, in quanto le operazioni asimmetriche risultano computazionalmente più impegnative di quelle simmetriche. La soluzione evita di assumere che una singola primitiva garantisca tutte le proprietà di sicurezza.

1.6 Setup iniziale e infrastruttura di fiducia

Prima dell'apertura dell'elezione, la Government Authority prepara l'infrastruttura di fiducia necessaria all'esecuzione del protocollo. Questa fase stabilisce quali entità infrastrutturali possono partecipare alla consultazione, quali chiavi pubbliche sono considerate valide, quali certificati sono accettati e quali parametri pubblici definiscono l'istanza elettorale.

Il setup comprende la generazione delle chiavi infrastrutturali, l'accreditamento delle entità operative, l'emissione dei certificati, la preparazione della Scrutiny Authority distribuita, la pubblicazione del manifesto dell'elezione e la gestione della lista di revoca. Al termine di questa fase, Voting Client, Voting Server, Registro Elettorale e nodi fiduciari della Scrutiny Authority dispongono delle informazioni necessarie per riconoscere le entità legittime e utilizzare chiavi pubbliche autentiche.

1.6.1 Generazione delle chiavi infrastrutturali

Ogni entità infrastrutturale partecipante genera autonomamente una propria coppia di chiavi asimmetriche. In particolare, generano una coppia di chiavi il Voting Server, il Registro Elettorale/Bulletin Board e ciascun nodo fiduciario SA_i della Scrutiny Authority distribuita.

Per una generica entità X, la coppia di chiavi viene indicata come:

$$(pk_X, sk_X).$$

La chiave pubblica pk_X viene inserita nella richiesta di accreditamento e, in caso di approvazione, nel certificato digitale emesso dalla Government Authority. La chiave privata sk_X rimane sotto il controllo dell'entità e viene usata per dimostrare il possesso della chiave pubblica corrispondente, autenticarsi nelle comunicazioni e firmare i messaggi prodotti nelle fasi successive del protocollo.

Il certificato dell'entità viene emesso e firmato dalla Government Authority tramite sk_{GA}. La firma dell'entità sulla richiesta di accreditamento serve invece come prova di possesso della chiave privata associata alla chiave pubblica da certificare.

1.6.2 Preparazione della Scrutiny Authority distribuita

La Scrutiny Authority è composta da un insieme di nodi fiduciari:

$$SA = \{SA_1, SA_2, \ldots, SA_n\}.$$

Prima dell'apertura dell'elezione viene fissata una soglia di cooperazione $\langle k, n\rangle$, dove n è il numero totale dei nodi fiduciari e k è il numero minimo di nodi necessari per completare lo scrutinio.

La chiave pubblica elettorale pk_{SA} viene pubblicata nei parametri dell'elezione e sarà usata dal Voting Client per cifrare la preferenza. La chiave privata di scrutinio associata a pk_{SA} è protetta all'interno dell'HSM: ogni nodo SA_i può contribuire ad una parte dell'autorizzazione e almeno k nodi devono cooperare per abilitare l'uso di sk_{SA} durante lo scrutinio finale.

Il protocollo utilizza questa organizzazione per ridurre il singolo punto di fiducia nella fase di scrutinio. Ai fini della soluzione proposta, si richiede che:

- ogni nodo SA_i sia accreditato e certificato dalla Government Authority;
- la soglia $\langle k, n\rangle$ sia pubblicata in $Params_{Election}$;
- nessun singolo nodo possa decifrare autonomamente le schede;
- durante lo scrutinio almeno k nodi cooperino per produrre il risultato finale.

Assunzione di fiducia sull'Hardware Security Module. Il protocollo assume dunque l'esistenza di un *Hardware Security Module* (HSM), incaricato della generazione della coppia elettorale (pk_{SA}, sk_{SA}), della custodia della chiave privata sk_{SA} e dell'esecuzione controllata delle operazioni crittografiche di scrutinio. Si assume che l'HSM sia un componente fidato (*trusted component*) e che soddisfi le seguenti proprietà:

- la coppia di chiavi elettorale (pk_{SA}, sk_{SA}) venga generata internamente al dispositivo;
- la chiave pubblica pk_{SA} sia esportabile e pubblicabile in $Params_{Election}$;
- la chiave privata sk_{SA} sia marcata come non esportabile e non venga mai rivelata in chiaro all'esterno dell'HSM;

- nessuna entità software esterna, inclusi i nodi della Scrutiny Authority, possa leggere direttamente il valore di sk_{SA};
- ogni operazione crittografica che richiede l'uso di sk_{SA} venga eseguita internamente all'HSM, che restituisce esclusivamente il risultato dell'operazione senza rivelare il segreto sottostante;
- l'HSM consenta l'utilizzo di sk_{SA} solo dopo aver verificato il rispetto della politica di autorizzazione prevista dal protocollo, ossia la presenza di almeno k approvazioni valide provenienti da nodi distinti della Scrutiny Authority;
- il dispositivo mantenga log affidabili delle operazioni effettuate e sia protetto contro accessi non autorizzati e manomissioni fisiche ragionevolmente prevedibili.

Si assume quindi che l'HSM non sia compromesso, che il suo firmware sia corretto e che le sue garanzie di isolamento e protezione siano effettivamente rispettate. L'eventuale compromissione del dispositivo rappresenta un rischio residuo del sistema e potrebbe consentire un uso non autorizzato di sk_{SA}, compromettendo la riservatezza delle schede e l'integrità della fase di scrutinio.

1.6.3 Richiesta di accreditamento

Il processo di accreditamento stabilisce quali componenti sono autorizzate a partecipare all'elezione e con quale ruolo. Le entità candidate, ovvero: Voting Server, Registro Elettorale e nodi fiduciari della Scrutiny Authority, inviano alla Government Authority una richiesta di certificazione.

La richiesta di accreditamento di una generica entità X è rappresentata da:

$$\begin{aligned}\text{RequestCert}_X = \Big(&ID_X, \text{Role}_X, ID_{\text{Election}}, pk_X,\\ &\text{OrganizationInfo}_X, \text{ValidFrom},\\ &\text{ValidTo}, \text{Nonce}_X, \sigma_X\Big)\end{aligned} \tag{1}$$

I campi principali hanno il seguente significato:

- ID_X: identificativo pubblico dell'entità candidata;
- $Role_X$: ruolo operativo richiesto, ad esempio VS, ER oppure SA_i;
- $ID_{Election}$: identificativo dell'elezione per cui si richiede l'accreditamento;
- pk_X: chiave pubblica oggetto della certificazione;
- $OrganizationInfo_X$: informazioni sull'infrastruttura che gestisce l'entità;
- $ValidFrom, ValidTo$: finestra temporale di validità richiesta;
- $Nonce_X$: valore casuale monouso usato per garantire freschezza della richiesta;
- σ_X: firma digitale prodotta con sk_X sui campi precedenti.

La firma della richiesta viene calcolata come:

$$\sigma_X = \mathrm{Sign}_{sk_X}\Big(H\big(ID_X \parallel \mathrm{Role}_X \parallel ID_{\mathrm{Election}} \parallel pk_X \parallel \mathrm{OrganizationInfo}_X \parallel \mathrm{ValidFrom} \parallel \mathrm{ValidTo} \parallel \mathrm{Nonce}_X\big)\Big) \tag{2}$$

Questa firma dimostra che l'entità che richiede il certificato possiede la chiave privata associata a pk_X.

1.6.4 Verifica della richiesta da parte della Government Authority

Ricevuta la richiesta, la Government Authority esegue i controlli necessari prima di autorizzare l'emissione del certificato. La verifica viene fatta attraverso questi controlli:

- controllo della presenza di ID_X nel registro $EligibleEntitiesDB$;
- controllo della coerenza tra $Role_X$ e il ruolo ammesso per l'entità candidata;
- controllo della corrispondenza con $ID_{Election}$;
- verifica della firma σ_X tramite pk_X, per accertare il possesso della chiave privata;
- controllo della finestra temporale $ValidFrom, ValidTo$;
- controllo della completezza e coerenza delle informazioni organizzative.

Il controllo su $EligibleEntitiesDB$ assume che tale registro sia stato popolato tramite una procedura amministrativa preliminare di accreditamento, esterna al protocollo crittografico ma necessaria per stabilire l'identità reale dell'entità candidata.

Se almeno uno dei controlli fallisce, la richiesta viene rifiutata e l'entità non può operare nel protocollo per quella specifica elezione. Se tutti i controlli hanno esito positivo, la Government Authority procede con l'emissione del certificato.

1.6.5 Emissione dei certificati

Per ogni entità accreditata, la Government Authority emette un certificato digitale che lega la chiave pubblica al ruolo operativo e all'istanza elettorale. Il certificato di una generica entità X è rappresentato da:

$$Cert_X = (ID_X, Role_X, ID_{Election}, pk_X, ValidFrom, ValidTo, SerialNumber_X, \sigma_{GA}^{Cert_X}).$$

Il campo $SerialNumber_X$ indica il numero seriale del certificato che identifica univocamente quest'ultimo.

La firma della Government Authority è calcolata sul contenuto del certificato esclusa la firma stessa:

$$\sigma_{GA}^{Cert_X} = Sign_{sk_{GA}}(H(ID_X \parallel Role_X \parallel ID_{Election} \parallel pk_X \parallel ValidFrom \parallel ValidTo \parallel SerialNumber_X)).$$

I certificati prodotti in questa fase sono

- $Cert_{VS}$, relativo al Voting Server;
- $Cert_{ER}$, relativo al Registro Elettorale/Bulletin Board;
- $Cert_{SA_i}$, relativo al nodo fiduciario SA_i della Scrutiny Authority.

Il certificato è un oggetto pubblico: può essere mostrato alle altre componenti dell'infrastruttura per dimostrare che la chiave pubblica dell'entità è stata approvata dalla Government Authority. L'entità che riceve il certificato può verificarne l'autenticità applicando la chiave pubblica della Government Authority:

$$Verify_{pk_{GA}}(Cert_X, \sigma_{GA}^{Cert_X}) = TRUE.$$

La Government Authority conserva una copia dei certificati emessi e del relativo stato, così da poter gestire successivamente eventuali revoche.

1.6.6 Pubblicazione del manifesto dell'elezione

Completato l'accreditamento delle entità, la Government Authority pubblica il manifesto dell'elezione, indicato come $Params_{Election}$. Il manifesto rappresenta lo stato comune della consultazione e consente a Voting Client, Voting Server, Registro Elettorale e nodi della Scrutiny Authority di riferirsi alla stessa istanza elettorale.

Il manifesto dell'elezione è rappresentato logicamente in questo modo:

$$\begin{aligned} Params_{Election} = (&ID_{Election}, ElectionType, Options, TimeOpen, TimeClose,\\ &Cert_{VS}, Cert_{ER}, \{Cert_{SA_i}\}_{i=1}^{n}, pk_{SA}, \langle k, n\rangle,\\ &H(Policy), Version, Timestamp, \sigma_{GA}^{Params}) \end{aligned}$$

dove:

$$\begin{aligned} \sigma_{GA}^{Params} = Sign_{sk_{GA}}(H(&ID_{Election} \parallel ElectionType \parallel Options \parallel TimeOpen \parallel TimeClose \parallel\\ &Cert_{VS} \parallel Cert_{ER} \parallel \{Cert_{SA_i}\}_{i=1}^{n} \parallel pk_{SA} \parallel \langle k, n\rangle \parallel\\ &H(Policy) \parallel Version \parallel Timestamp)). \end{aligned}$$

I campi del manifesto sono

- $ID_{Election}$: identificativo univoco dell'elezione;
- $ElectionType$: tipo di consultazione;
- $Options$: insieme delle opzioni ammesse, nel caso specifico $\{\text{Sì}, \text{No}\}$;
- $TimeOpen, TimeClose$: istanti di apertura e chiusura della votazione;
- $Cert_{VS}, Cert_{ER}, \{Cert_{SA_i}\}_{i=1}^{n}$: certificati delle entità operative accreditate;
- pk_{SA}: chiave pubblica elettorale usata per cifrare la preferenza;
- $\langle k, n\rangle$: parametri della Scrutiny Authority distribuita;

- $H(Policy)$: digest delle regole dell'elezione;
- $Version, Timestamp$: versione e istante di pubblicazione del manifesto;
- σ_{GA}^{Params}: firma della Government Authority sul manifesto.

Il Voting Client e le altre entità infrastrutturali verificano la firma della Government Authority prima di utilizzare $Params_{Election}$. Inoltre, controllano che i certificati contenuti nel manifesto siano coerenti con i ruoli previsti e che non risultino revocati.

1.6.7 Pubblicazione e aggiornamento della lista di revoca

La Government Authority mantiene e pubblica la lista dei certificati revocati, indicata come $CRL_{Election}$. La lista contiene i numeri seriali dei certificati che non devono più essere accettati per l'elezione corrente, ad esempio in caso di compromissione della chiave privata o rimozione dell'entità dall'infrastruttura accreditata.

La lista di revoca è rappresentata come:

$$\begin{aligned}\mathrm{CRL}_{\mathrm{Election}} = \Big(&ID_{\mathrm{Election}}, \mathrm{RevokedSerialNumbers}, \mathrm{Version}_{\mathrm{CRL}}, \\ &\mathrm{Timestamp}_{\mathrm{CRL}}, \mathrm{NextUpdate}, \sigma_{\mathrm{GA}}^{\mathrm{CRL}}\Big)\end{aligned} \tag{3}$$

La firma della Government Authority è calcolata come:

$$\begin{aligned}\sigma_{\mathrm{GA}}^{\mathrm{CRL}} = \mathrm{Sign}_{sk_{\mathrm{GA}}}\Big(H\Big(&ID_{\mathrm{Election}} \parallel \mathrm{RevokedSerialNumbers} \parallel \\ &\mathrm{Version}_{\mathrm{CRL}} \parallel \mathrm{Timestamp}_{\mathrm{CRL}} \parallel \mathrm{NextUpdate}\Big)\Big)\end{aligned} \tag{4}$$

I campi principali hanno il seguente significato:

- $ID_{Election}$: associa la lista di revoca all'elezione corrente;
- $RevokedSerialNumbers$: insieme dei numeri seriali dei certificati revocati;
- $Version_{CRL}$: versione della lista, usata per rilevare configurazioni obsolete;
- $Timestamp_{CRL}$: istante di emissione della lista;
- $NextUpdate$: istante entro cui deve essere disponibile una versione aggiornata;
- σ_{GA}^{CRL}: firma della Government Authority sulla lista.

Prima di accettare un certificato, una componente verifica la firma della CRL, controlla che la lista sia riferita a $ID_{Election}$, verifica che la versione sia aggiornata e controlla che il seriale del certificato non compaia in $RevokedSerialNumbers$.

La distribuzione di $Params_{Election}$, dei certificati e di $CRL_{Election}$ avviene secondo un modello di *pull*: le entità del protocollo recuperano la versione più recente dal punto di

pubblicazione della Government Authority o da un mirror pubblico autorizzato. Ogni oggetto recuperato viene accettato solo se la firma della Government Authority è valida.

Il Voting Client effettua il pull di $Params_{Election}$ e $CRL_{Election}$ all'avvio dell'applicazione e prima dell'apertura del canale sicuro con il Voting Server. Il Voting Server e il Registro Elettorale aggiornano la CRL prima di accettare messaggi firmati da altre entità infrastrutturali. I nodi della Scrutiny Authority recuperano la CRL prima di partecipare allo scrutinio finale.

Per ragioni di efficienza, le entità possono mantenere in cache una versione firmata di $Params_{Election}$ e $CRL_{Election}$, purché controllino versione, timestamp e finestra di validità.

La soluzione adotta un comportamento di tipo hard-fail: se una componente non riesce a recuperare o verificare una versione valida e aggiornata di $CRL_{Election}$, il certificato non viene accettato e l'operazione viene interrotta. Questa scelta privilegia la sicurezza rispetto alla disponibilità del servizio.

1.6.8 Revoca dell'accreditamento

La revoca dell'accreditamento permette alla Government Authority di invalidare un certificato precedentemente emesso. La revoca può avvenire su richiesta dell'entità interessata, ad esempio in caso di sospetta compromissione della chiave privata, oppure d'ufficio, se la Government Authority rileva che una componente non deve più partecipare al protocollo.

Nel caso di richiesta volontaria, l'entità invia alla Government Authority un messaggio contenente almeno l'identificativo dell'elezione, il numero seriale del certificato da revocare, una motivazione e un timestamp:

$$RevocationRequest_X = (ID_{Election}, SerialNumber_X, Reason_X, Timestamp_X, \sigma_X^{Rev}).$$

La firma σ_X^{Rev} viene prodotta con la chiave privata dell'entità:

$$\sigma_X^{Rev} = Sign_{sk_X}(H(ID_{Election} \parallel SerialNumber_X \parallel Reason_X \parallel Timestamp_X)).$$

La Government Authority verifica la richiesta, controlla che il certificato sia stato effettivamente emesso per l'entità indicata, controllandone la firma attraverso la chiave pubblica salvata. Dopo aver effettuato il controllo, viene aggiornato lo stato del certificato e aggiorna il proprio registro interno.

Nel caso di revoca d'ufficio, la Government Authority aggiorna direttamente lo stato del certificato associato all'entità, utilizzando le informazioni conservate durante la fase di accreditamento. In entrambi i casi, la revoca diventa operativa tramite la pubblicazione di una nuova versione di $CRL_{Election}$, con $Version_{CRL}$ incrementata e firma aggiornata.

Al termine del setup, l'infrastruttura dispone di entità accreditate, certificati verificabili, parametri pubblici firmati e una lista di revoca controllabile. Da questo momento il protocollo può passare alle fasi operative di autenticazione dell'elettore, rilascio del token e raccolta della scheda cifrata.

1.6.9 Premessa sui canali di comunicazione sicuri

Nel seguito si assume che tutte le comunicazioni sensibili tra le entità del sistema avvengano tramite un canale sicuro autenticato. La procedura descritta in dettaglio nella Sezione 1.12 per il collegamento tra Voting Client e Voting Server rappresenta il caso principale del protocollo, ma la stessa logica viene assunta anche per gli altri scambi sensibili tra componenti, adattando il certificato e il ruolo dell'entità destinataria.

In particolare, prima di accettare una comunicazione con una generica entità X, il mittente verifica che:

- il certificato $Cert_X$ sia firmato dalla Government Authority;
- il certificato sia valido per $ID_{Election}$;
- il ruolo contenuto nel certificato sia coerente con l'operazione richiesta;
- il certificato non sia scaduto;
- il certificato non compaia in $CRL_{Election}$;
- la sessione utilizzi valori freschi, come nonce o timestamp, per ridurre il rischio di replay.

Solo dopo questa verifica viene stabilita una chiave simmetrica di sessione $k_{session}$, usata dallo strato di comunicazione per garantire confidenzialità, integrità, autenticazione dei messaggi e freschezza della sessione.

Di conseguenza, quando nelle fasi successive si indica che un messaggio viene trasmesso tramite canale sicuro, si intende che tale canale è stato aperto secondo la logica descritta nella Sezione 1.12: verifica del certificato, controllo della lista di revoca, autenticazione dell'entità remota, key exchange autenticato e uso di una chiave simmetrica di sessione.

Questa assunzione protegge i messaggi in transito da intercettazione, modifica, impersonificazione e replay di sessione. Non rende però pienamente fidato il dispositivo dell'elettore: se il client locale è compromesso, un attaccante potrebbe alterare i dati prima dell'invio o conservare informazioni utili alla coercizione.

1.7 Flusso generale del protocollo

Dopo il completamento del setup iniziale, il protocollo entra nella fase operativa dell'elezione. In questa fase l'elettore usa il Voting Client per autenticarsi, ottenere un token di autorizzazione, compilare la scheda, inviare il voto cifrato e ricevere una ricevuta utile alla verifica di inclusione.

Il flusso complessivo può essere riassunto come segue:

Fase	Nome	Descrizione sintetica
1	Inizializzazione del Voting Client	Il client recupera e verifica i parametri pubblici dell'elezione, la lista di revoca e i certificati delle entità.

Fase	Nome	Descrizione sintetica
2	Autenticazione dell'elettore	L'elettore viene autenticato dalla Government Authority tramite il modulo di identità digitale.
3	Verifica di idoneità e rilascio del token	La Government Authority verifica il diritto al voto e rilascia un token di autorizzazione firmato.
4	Preparazione locale della scheda	Il Voting Client acquisisce la preferenza, cifra la scheda con pk_{SA} e prepara la busta di voto.
5	Apertura del canale sicuro verso il Voting Server	Il client verifica $Cert_{VS}$, controlla la CRL e stabilisce un canale autenticato, cifrato e integro.
6	Invio e verifica della busta di voto	Il Voting Server verifica token, prova di possesso e unicità del voto.
7	Pubblicazione sul Registro Elettorale	Il Voting Server inoltra la scheda cifrata al Registro Elettorale/Bulletin Board, che pubblica il record e produce una ricevuta.
8	Scrutinio finale	Alla chiusura delle urne, la Scrutiny Authority distribuita coopera per decifrare e conteggiare le schede accettate.
9	Pubblicazione e verifica	Vengono pubblicati risultato, record finali e dati necessari alla verificabilità individuale e universale.

Le fasi sono descritte assumendo che le parti oneste seguano il protocollo. Eventuali deviazioni, compromissioni o comportamenti malevoli vengono considerate nell'analisi di sicurezza.

1.8 Fase 1: inizializzazione del Voting Client

Quando l'elettore apre l'applicazione ufficiale, il Voting Client inizializza il contesto locale dell'elezione. Il client recupera $Params_{Election}$ e $CRL_{Election}$, quindi verifica che entrambi siano firmati dalla Government Authority.

La verifica dei parametri pubblici avviene controllando la firma:

$$Verify_{pk_{GA}}(Params_{Election}, \sigma_{GA}^{Params}) = TRUE.$$

Analogamente, il client verifica la lista di revoca:

$$Verify_{pk_{GA}}(CRL_{Election}, \sigma_{GA}^{CRL}) = TRUE.$$

Il Voting Client controlla poi che:

- $ID_{Election}$ corrisponda all'elezione selezionata;

- la consultazione sia all'interno della finestra temporale ammessa;
- i certificati $Cert_{VS}$, $Cert_{ER}$ e $\{Cert_{SA_i}\}_{i=1}^{n}$ siano coerenti con i ruoli dichiarati;
- nessuno dei certificati compaia in $CRL_{Election}$;
- la chiave pubblica elettorale pk_{SA} sia presente nel manifesto firmato.

Successivamente, il client genera o recupera la coppia di chiavi dell'elettore per la specifica elezione:

$$(pk_U, sk_U).$$

La coppia (pk_U, sk_U) è associata a $ID_{Election}$. La chiave pubblica pk_U sarà usata per vincolare il token di autorizzazione al client dell'elettore, mentre sk_U rimane sotto il controllo dell'utente e non viene trasmessa.

1.9 Fase 2: autenticazione dell'elettore

Premessa. La comunicazione tra Voting Client e Government Authority avviene tramite un canale sicuro autenticato, secondo l'assunzione generale descritta nella Sezione 1.6.9. In particolare, si assume che il client comunichi con una Government Authority autenticata e che i messaggi scambiati siano protetti in confidenzialità, integrità e freschezza.

Dopo l'inizializzazione, l'elettore avvia la procedura di autenticazione presso la Government Authority. Questa fase è svolta dal modulo IP, che verifica l'identità reale dell'utente secondo il meccanismo di identità digitale previsto dal sistema.

Il Voting Client invia alla Government Authority una richiesta di autenticazione contenente l'identificativo dell'elezione, la chiave pubblica generata per l'elezione e valori di freschezza della sessione:

$$AuthRequest_U = (ID_{Election}, pk_U, nonce_U, timestamp_U).$$

Il modulo IP autentica l'elettore e produce un'asserzione di autenticazione, firmata dalla Government Authority:

$$AuthAssertion_U = (ID_U, ID_{Election}, pk_U, nonce_U, timestamp_{GA}, \sigma_{GA}^{Auth}).$$

La firma σ_{GA}^{Auth} è calcolata sui campi dell'asserzione:

$$\sigma_{GA}^{Auth} = Sign_{sk_{GA}}(H(ID_U \parallel ID_{Election} \parallel pk_U \parallel nonce_U \parallel timestamp_{GA})).$$

Questa fase stabilisce l'identità reale dell'elettore nei confronti della Government Authority. Non implica ancora il rilascio del diritto al voto: l'autenticazione viene seguita dalla verifica dell'idoneità elettorale.

1.10 Fase 3: verifica dell'idoneità e rilascio del token

Ricevuta l'asserzione di autenticazione, il modulo EV della Government Authority verifica che l'elettore autenticato sia presente in $EligibleUsersDB$ e sia abilitato a partecipare alla consultazione identificata da $ID_{Election}$.

Se la verifica fallisce, il protocollo termina senza emissione del token. Se la verifica ha esito positivo, il modulo VTI genera un permesso di voto:

$$Permit_U = (ID_{Election}, SerialNumber_{Token}, pk_U, ValidFrom, ValidTo).$$

Il permesso contiene l'identificativo dell'elezione, un numero seriale univoco del token, la chiave pubblica dell'elettore e una finestra temporale di validità. Non contiene la preferenza dell'elettore.

La Government Authority firma il permesso:

$$\sigma_{GA}^{Permit} = Sign_{sk_{GA}}(H(ID_{Election} \parallel SerialNumber_{Token} \parallel pk_U \parallel ValidFrom \parallel ValidTo)).$$

Il token di autorizzazione è quindi:

$$VotingToken_U = (Permit_U, \sigma_{GA}^{Permit}).$$

Per consegnare il token al Voting Client, la Government Authority lo cifra usando la chiave pubblica dell'elettore:

$$EncToken_U = Enc_{pk_U}(VotingToken_U).$$

Il Voting Client recupera il token decifrando localmente:

$$VotingToken_U = Dec_{sk_U}(EncToken_U).$$

La Government Authority aggiorna $IssuedTokensDB$ in maniera atomica, registrando $EncToken_U$ e che per l'elettore autenticato è stato emesso un token per $ID_{Election}$. Questa registrazione serve a impedire l'emissione di più token per lo stesso elettore nella medesima elezione.

1.11 Fase 4: preparazione locale della scheda

Dopo aver ottenuto il token, il Voting Client mostra all'elettore le opzioni ammesse dalla consultazione, contenute in $Params_{Election}$. Nel caso base del referendum binario:

$$Options = \{\text{Sì}, \text{No}\}.$$

L'elettore seleziona una preferenza:

$$v \in Options.$$

Il Voting Client genera un valore casuale $nonce_{ballot}$, usato per legare la scheda a una specifica esecuzione locale del protocollo.

Prima dell'invio della scheda, il Voting Client cifra localmente la preferenza selezionata utilizzando la chiave pubblica elettorale della Scrutiny Authority, pubblicata in $Params_{Election}$. Poiché nel caso considerato il voto è binario, la cifratura della preferenza non deve essere deterministica. Se lo stesso valore v producesse sempre lo stesso ciphertext sotto la stessa chiave pubblica pk_{SA}, un osservatore del Bulletin Board potrebbe riconoscere schede uguali e ricavare informazioni statistiche dai pattern dei ciphertext pubblicati. Per questo motivo il protocollo richiede uno schema di cifratura asimmetrica probabilistica, ad esempio RSA-OAEP, in modo che due cifrature della stessa preferenza producano ciphertext diversi grazie alla randomizzazione interna dello schema. Il Voting Client calcola quindi:

$$EncVoto_{SA} = Enc^{prob}_{pk_{SA}}(v),$$

dove Enc^{prob} indica una cifratura probabilistica della preferenza. Il Voting Server e il Registro Elettorale trattano $EncVoto_{SA}$ come oggetto opaco: possono registrarne, firmarne o pubblicarne il riferimento, ma non possono ricavare la preferenza in chiaro.

Da questo momento, Voting Server e Registro Elettorale trattano solo la scheda cifrata. La preferenza in chiaro rimane disponibile nel Voting Client solo durante la compilazione locale e deve essere eliminata dalla memoria non appena non risulta più necessaria.

Dopo aver prodotto la scheda cifrata, il Voting Client calcola un commitment crittografico della scheda:

$$BallotCommit_U = H(ID_{Election} \parallel EncVoto_{SA} \parallel nonce_{ballot}).$$

Il valore $BallotCommit_U$ non rivela la preferenza, poiché viene calcolato su una scheda cifrata probabilisticamente. Il Voting Client conserva localmente $BallotCommit_U$ fino alla verifica individuale finale, in modo da poterlo confrontare con il valore contenuto nella ricevuta firmata dal Registro Elettorale.

Il commitment non impedisce da solo la perfetta integrità della scheda. La sua funzione è rendere rilevabile dall'elettore una sostituzione della scheda cifrata prima della pubblicazione.

Il client prepara inoltre una firma di possesso della chiave privata associata al token:

$$\sigma_U^{Ballot} = Sign_{sk_U}(H(ID_{Election} \parallel EncVoto_{SA} \parallel VotingToken_U \parallel nonce_{ballot} \parallel BallotCommit_U)).$$

Questa firma consente al Voting Server di verificare che chi presenta il token possieda la chiave privata corrispondente a pk_U, presente nel permesso firmato dalla Government Authority.

1.12 Fase 5: verifica del Voting Server e apertura del canale sicuro

Prima di trasmettere la busta di voto al Voting Server, il Voting Client deve verificare che il destinatario sia effettivamente un'entità autorizzata per l'elezione corrente. Questa fase non riguarda l'autenticazione dell'elettore, già avvenuta presso la Government Authority durante il rilascio del token, ma l'autenticazione dell'entità infrastrutturale incaricata di ricevere le richieste di voto.

Il Voting Client utilizza $Params_{Election}$ e $CRL_{Election}$, pubblicati e firmati dalla Government Authority. In particolare, recupera il certificato del Voting Server $Cert_{VS}$ e verifica che esso sia stato emesso dalla Government Authority:

$$Verify_{pk_{GA}}(Cert_{VS}, \sigma_{GA}^{Cert_{VS}}) = TRUE.$$

Il client controlla inoltre che il certificato:

- sia valido per l'elezione identificata da $ID_{Election}$;
- sia associato al ruolo di Voting Server;
- non sia scaduto;
- non compaia in $CRL_{Election}$.

Questa verifica realizza il binding tra identità infrastrutturale, ruolo e chiave pubblica. Il Voting Client accetta pk_{VS} solo se tale chiave è contenuta in un certificato valido e firmato dalla Government Authority. In questo modo, la chiave pubblica del server non viene assunta come affidabile perché ricevuta dalla rete, ma perché certificata dall'autorità responsabile dell'infrastruttura elettorale.

Superata la verifica, il Voting Client estrae dal certificato la chiave pubblica del Voting Server:

$$Cert_{VS} \longrightarrow pk_{VS}.$$

A questo punto il Voting Client apre un canale sicuro autenticato con il Voting Server:

$$VC \leftrightarrow VS : \text{SecureChannel}(Cert_{VS}, ID_{Election}).$$

Il canale deve garantire autenticazione del server, confidenzialità dei messaggi, integrità dei dati trasmessi e freschezza della sessione. La freschezza è ottenuta tramite nonce, così da evitare il riutilizzo di messaggi appartenenti a sessioni precedenti.

Il canale sicuro può essere istanziato mediante un key exchange autenticato basato su Diffie-Hellman. In tale scambio, Voting Client e Voting Server contribuiscono entrambi alla derivazione di un segreto condiviso di sessione; tuttavia, i valori pubblici usati nello scambio non vengono accettati in modo cieco. Durante l'apertura del canale sicuro, il Voting Server dimostra di essere il titolare della chiave pubblica contenuta in $Cert_{VS}$. In particolare, il server usa la propria chiave privata sk_{VS} per autenticare lo scambio

di sessione. Poiché il Voting Client ha già verificato che $Cert_{VS}$ sia stato firmato dalla Government Authority, che non sia revocato e che sia valido per $ID_{Election}$, il client può accettare la chiave di sessione derivata come chiave condivisa con il Voting Server autorizzato. In questo modo, il key exchange produce una chiave simmetrica associata al server certificato per l'elezione corrente:

$$k_{session}.$$

La chiave $k_{session}$ viene usata dallo strato di comunicazione sicura per proteggere i messaggi successivi tra Voting Client e Voting Server. Ai fini del protocollo di voto, è sufficiente assumere che il canale stabilito fornisca:

$$\text{autenticazione del server} \ \wedge \ \text{confidenzialità} \ \wedge \ \text{integrità} \ \wedge \ \text{freschezza}.$$

Terminata questa fase, il Voting Client dispone di un canale sicuro verso un Voting Server autenticato per $ID_{Election}$. Solo a questo punto il client trasmette la busta di voto contenente il token di autorizzazione e la scheda cifrata.

1.13 Fase 6: invio della busta di voto e verifica del token

Il Voting Client costruisce la busta di voto:

$$Ballot = (ID_{Election}, EncVoto_{SA}, VotingToken_U, nonce_{ballot}, BallotCommit_U, \sigma_U^{Ballot}).$$

Il messaggio viene inviato al Voting Server attraverso il canale sicuro stabilito nella Fase 5, secondo l'assunzione generale descritta nella Sezione 1.6.9:

$$M_{vote} : VC \rightarrow VS : Ballot.$$

Il Voting Server esegue i controlli necessari prima di accettare la scheda:

- verifica che $\text{ID}_{\text{Election}}$ corrisponda all'elezione corrente;
- verifica la firma della Government Authority sul token;
- controlla la validità temporale di Permit_U;
- estrae pk_U dal permesso firmato;
- verifica σ_U^{Ballot} usando pk_U;
- verifica la coerenza di BallotCommit_U andandolo a ricalcolare con i dati in chiaro;
- controlla che il token non sia già stato usato;
- verifica la freschezza del messaggio tramite $\text{nonce}_{\text{ballot}}$.

Per ridurre il carico computazionale in presenza di richieste malformate o malevole, il Voting Server esegue i controlli in ordine crescente di costo. Vengono prima verificati formato del messaggio, $ID_{Election}$, finestra temporale e presenza dei campi obbligatori; solo successivamente vengono eseguite le verifiche crittografiche più costose, come la verifica delle firme.

La verifica della firma della Government Authority sul token è:

$$Verify_{pk_{GA}}(Permit_U, \sigma_{GA}^{Permit}) = TRUE.$$

La verifica della prova di possesso della chiave dell'elettore, serve ad assicurarsi che il Token all'interno del Ballot sia stato utilizzato dall'effettivo richiedente dell'autorizzazione al voto. Questo perché, a partire dalla generazione delle chiavi pubbliche e private da parte del client, l'autorizzazione al voto rilasciata dalla GA è firmata dalla suddetta e contiene anche la chiave pubblica dell'utente che la richiede. La verifica viene effettuata in questo modo:

$$\begin{aligned}\text{Verify}_{pk_U}\Big(&ID_{\text{Election}} \parallel \text{EncVoto}_{\text{SA}} \parallel \text{VotingToken}_U \parallel \text{nonce}_{\text{ballot}},\\ &\text{BallotCommit}_U, \sigma_U^{\text{Ballot}}\Big) = \text{TRUE}\end{aligned} \tag{5}$$

Se almeno uno dei controlli fallisce, il Voting Server rifiuta la busta di voto. Se tutti i controlli hanno esito positivo, il Server procede con la registrazione del Token.

1.14 Fase 7: registrazione del token speso

Per garantire l'unicit'a del voto, il Voting Server deve impedire il riutilizzo dello stesso token di autorizzazione. Il server non conserva il token completo in chiaro all'interno di (SpentTokensDB), nè memorizza direttamente il numero seriale del token. Al contrario, salva un tag deterministico di lookup calcolato a partire dal numero seriale e da una chiave interna derivata dal segreto del Voting Server.

Per la specifica elezione, il Voting Server deriva innanzitutto una chiave interna:

$$K_{\text{Spent}}^{\text{Election}} = H(K_{\text{Master_Server}} \parallel ID_{\text{Election}})$$

Il valore $K_{\text{Spent}}^{\text{Election}}$ non è un sale classico casuale per record, ma una chiave deterministica specifica per l'elezione, derivata da un segreto interno del Voting Server. La sua funzione è permettere il calcolo di un riferimento non invertibile del token, evitando che il contenuto di *SpentTokensDB* riveli direttamente i numeri seriali dei token utilizzati.

A partire dal numero seriale del token, il Voting Server calcola quindi:

$$LookupTag_{\text{Token}} = \text{HMAC}_{K_{\text{Spent}}^{\text{Election}}}(SerialNumber_{\text{Token}})$$

Il valore $LookupTag_{\text{Token}}$ viene cercato in *SpentTokensDB*. Se il tag è già presente in stato *pending* o *committed*, il token è considerato già utilizzato e la busta di voto viene rifiutata.

Se il tag non è presente, il Voting Server inserisce una nuova entry in maniera atomica (evitando problemi con richieste concorrenti) nello stato *pending*:

$$SpentEntry = (LookupTag_{\text{Token}}, ID_{\text{Election}}, \text{status}, \text{timestamp}_{\text{VS}})$$

Lo stato *pending* indica che il token è stato validato dal Voting Server e temporaneamente bloccato contro tentativi di riuso, ma che la scheda cifrata non 'e ancora stata confermata dal Registro Elettorale.

Il token viene considerato definitivamente consumato solo dopo la ricezione e la verifica della ricevuta firmata ($Receipt_U$), prodotta dal Registro Elettorale. Solo in quel momento il Voting Server aggiorna l'entry corrispondente:

$$\text{status} = \text{committed}$$

Questa scelta evita che un token venga marcato come definitivamente speso prima che esista una prova firmata dell'effettiva presa in carico della scheda cifrata da parte del Registro Elettorale.

Inoltre, poiché lo stesso $SerialNumber_{\text{Token}}$ produce sempre lo stesso $LookupTag_{\text{Token}}$ per la stessa elezione, il Voting Server può rilevare tentativi di riuso senza memorizzare il token o il suo seriale in chiaro.

Lo stato *pending* indica che il token è stato validato dal Voting Server e temporaneamente bloccato contro tentativi di riuso, ma che la scheda cifrata non è ancora stata confermata dal Registro Elettorale. Questa scelta evita che un token venga marcato come definitivamente speso prima che esista una prova firmata dell'effettiva presa in carico della scheda da parte del Registro Elettorale.

1.15 Fase 8: pubblicazione sul Registro Elettorale

Dopo aver accettato la busta di voto e aver marcato il token in stato *pending*, il Voting Server inoltra al Registro Elettorale/Bulletin Board solo la scheda cifrata e i metadati necessari alla registrazione pubblica. Il token di autorizzazione non viene pubblicato sul registro, in modo da ridurre il rischio di correlazione tra autorizzazione al voto e scheda registrata.

Il payload del messaggio inviato dal Voting Server al Registro Elettorale è:

$$M_{VS} = (ID_{Election}, EncVoto_{SA}, timestamp_{VS}, BallotCommit_U, nonce_{ballot}).$$

Il Voting Server firma tale payload:

$$\sigma_{VS}^{M} = Sign_{sk_{VS}}(H(M_{VS})).$$

La trasmissione tra Voting Server e Registro Elettorale avviene tramite un canale sicuro autenticato, secondo l'assunzione generale descritta nella Sezione 1.6.9.

Il messaggio inviato al Registro Elettorale è quindi:

$$M_{board} : VS \rightarrow ER : (M_{VS}, \sigma_{VS}^{M}).$$

Il Registro Elettorale verifica la firma del Voting Server tramite il certificato $Cert_{VS}$, controllando che il certificato sia valido, riferito a $ID_{Election}$, associato al ruolo di Voting Server e non revocato in $CRL_{Election}$.

Il Registro Elettorale verifica inoltre la coerenza interna tra la scheda cifrata e il commitment ricevuto:

$$BallotCommit_U = H(ID_{Election} \parallel EncVoto_{SA} \parallel nonce_{ballot}).$$

Se il controllo fallisce, il record viene rifiutato. Questo controllo permette al Registro di rilevare messaggi incoerenti, ma non impedisce a un Voting Server malevolo di sostituire sia $EncVoto_{SA}$ sia $BallotCommit_U$. In tale caso, la sostituzione resta rilevabile dall'elettore tramite il confronto con il commitment conservato localmente.

Se la verifica ha esito positivo, il Registro inserisce M_{board} nella propria Bulletin Board append-only, assegnandogli una posizione pubblica i e un timestamp di registrazione:

$$BoardRecord_i = (i, M_{board}, timestamp_{ER}).$$

L'indice i non rappresenta l'identità dell'elettore e non contiene informazioni sulla preferenza espressa. Esso indica soltanto la posizione del record nella sequenza pubblica del Bulletin Board e, di conseguenza, la posizione della foglia nella struttura Merkle finale.

Oltre alla registrazione append-only, il Registro Elettorale organizza i record tramite una struttura Merkle-based. Questa struttura viene usata per fornire, alla fine dell'elezione, una prova di inclusione della scheda cifrata nel registro pubblicato.

Per ogni $BoardRecord_i$, il Registro Elettorale calcola la foglia associata:

$$Leaf_i = H(BoardRecord_i).$$

La foglia $Leaf_i$ viene inserita nel Merkle Tree del Registro Elettorale. Poiché durante la fase di voto la struttura dell'albero può cambiare a seguito dell'inserimento di nuove schede, il Registro non fornisce ancora un Merkle path definitivo. La ricevuta immediata contiene quindi soltanto un riferimento crittografico firmato alla scheda cifrata accettata e non un riferimento al voto stesso:

$$Receipt_U = (ID_{Election}, Leaf_i, timestamp_{ER}, BallotCommit_U, \sigma_{ER}^{Receipt}).$$

La firma del Registro Elettorale è calcolata come:

$$\sigma_{ER}^{Receipt} = Sign_{sk_{ER}}\Big(H(ID_{Election} \parallel Leaf_i \parallel timestamp_{ER} \parallel BallotCommit_U)\Big).$$

La ricevuta viene restituita al Voting Server tramite il canale sicuro autenticato tra Registro Elettorale e Voting Server:

$$M_{receipt} : ER \rightarrow VS : Receipt_U.$$

Alla ricezione di $Receipt_U$, il Voting Server verifica la firma del Registro Elettorale:

$$Verify_{pk_{ER}}(ID_{Election} \parallel Leaf_i \parallel timestamp_{ER} \parallel BallotCommit_U, \sigma_{ER}^{Receipt}) = TRUE.$$

Se la verifica ha esito positivo, il Voting Server aggiorna la corrispondente entry in $SpentTokensDB$:

$$status = committed.$$

Lo stato *committed* indica che il token è stato consumato in modo definitivo e che esiste una ricevuta firmata associata alla scheda cifrata pubblicata dal Registro Elettorale. Se invece la ricevuta non viene ricevuta, oppure la firma del Registro Elettorale non risulta valida, l'entry rimane in stato *pending* e viene trattata come anomalia.

Il Voting Server inoltra quindi la ricevuta al Voting Client attraverso il canale sicuro precedentemente stabilito nella Fase 5:

$$M_{receipt}^{VC} : VS \rightarrow VC : Receipt_U.$$

Il Voting Client mantiene aperto il canale sicuro con il Voting Server fino alla ricezione di $Receipt_U$. Il Voting Client, alla ricezione della ricevuta, verifica che $BallotCommit_U$ coincida con il valore calcolato e conservato localmente durante la Fase 4. Se i valori non coincidono, il client segnala una possibile sostituzione della scheda cifrata. Solo dopo aver ricevuto la ricevuta firmata dal Registro Elettorale e verificato il commitment, il client considera conclusa la fase di invio del voto.

La ricevuta consente all'elettore di conservare una prova firmata di presa in carico della scheda cifrata da parte del Registro Elettorale. Essa non costituisce ancora una Proof of Membership completa, poiché il Merkle Tree non ha una radice finale definitiva durante lo svolgimento dell'elezione. La prova di inclusione completa viene resa verificabile solo alla chiusura dell'elezione, quando il Registro pubblica lo stato finale del Bulletin Board e la radice Merkle finale.

1.16 Fase 9: chiusura delle urne e scrutinio finale

Alla chiusura della finestra temporale di voto, il Registro Elettorale cristallizza l'insieme delle schede cifrate accettate. Da questo momento non vengono più accettati nuovi record per l'elezione corrente.

Il Registro Elettorale pubblica lo stato finale del Bulletin Board:

$$BB_{final} = \{BoardRecord_i\}_{i=1}^{m},$$

dove m indica il numero totale di schede cifrate accettate e pubblicate nel registro finale.

A partire dai record finali, il Registro calcola e pubblica la radice finale della struttura Merkle:

$$Root_{final}.$$

La radice finale viene firmata dal Registro Elettorale:

$$\sigma_{ER}^{Root} = Sign_{sk_{ER}}\Big(H(ID_{Election} \parallel Root_{final} \parallel timestamp_{close})\Big).$$

Il Registro rende inoltre disponibile, per ciascuna foglia, il relativo Merkle path definitivo:

$$GetProof(Leaf_i) \rightarrow MerklePath_i^{final}.$$

In questo modo, la fase di registrazione viene chiusa in modo verificabile: i record pubblicati nel Bulletin Board finale non possono essere modificati senza cambiare la radice finale firmata.

Terminata la pubblicazione del Bulletin Board finale, la Scrutiny Authority distribuita opera esclusivamente sui record presenti in BB_{final}. Da ciascun $BoardRecord_i$ viene estratta la scheda cifrata $EncVoto_{SA}^{(i)}$, contenuta nel payload M_{VS} firmato dal Voting Server.

Prima di procedere allo scrutinio, i nodi della Scrutiny Authority verificano:

- la firma del Registro Elettorale sulla radice finale $Root_{final}$;
- la coerenza dei record pubblicati con la struttura Merkle finale;
- la firma del Voting Server sui messaggi M_{board};
- la validità dei certificati delle entità coinvolte rispetto a $Params_{Election}$ e $CRL_{Election}$.

Sia:

$$C = \{EncVoto_{SA}^{(1)}, EncVoto_{SA}^{(2)}, \ldots, EncVoto_{SA}^{(m)}\}$$

l'insieme delle schede cifrate estratte dal Bulletin Board finale.

Lo scrutinio viene eseguito tramite una procedura congiunta della Scrutiny Authority. Ogni nodo fiduciario SA_j, con $j \in \{1, \ldots, n\}$, verifica preliminarmente la coerenza del Bulletin Board finale e decide se autorizzare lo scrutinio.

Se il nodo approva la procedura, produce una autorizzazione firmata:

$$Approval_j = (ID_{Election}, Root_{final}, m, timestamp_{SA_j}).$$

La firma del nodo è:

$$\sigma_{SA_j}^{Approval} = Sign_{sk_{SA_j}}\Big(H(ID_{Election} \parallel Root_{final} \parallel m \parallel timestamp_{SA_j})\Big).$$

L'autorizzazione pubblicabile è quindi:

$$PublishedApproval_j = (Approval_j, \sigma_{SA_j}^{Approval}).$$

Un' autorizzazione viene accettata solo se proviene da un nodo SA_j indicato in $Params_{Election}$, se il certificato $Cert_{SA_j}$ è valido e non revocato, e se la firma del nodo è corretta. Si assume che quando sono disponibili almeno k autorizzazioni valide prodotte da nodi distinti, la procedura di scrutinio viene abilitata. A questo punto la chiave privata di scrutinio sk_{SA}, custodita all'interno dell'Hardware Security Module, viene usata internamente per decifrare le schede cifrate presenti in BB_{final}.

Per ciascuna scheda cifrata $EncVoto_{SA}^{(i)}$, la decifratura è:

$$v_i = Dec_{sk_{SA}}(EncVoto_{SA}^{(i)}).$$

Nel caso proposto, il valore ottenuto deve appartenere all'insieme:

$$v_i \in Options = \{\text{Sì}, \text{No}\}.$$

Se la decifratura di una scheda fallisce oppure non produce un valore appartenente alle opzioni ammesse, la scheda viene marcata come non valida ai fini del conteggio.

Il risultato finale viene calcolato aggregando tutte le preferenze valide:

$$Tally = (count_{\text{Sì}}, count_{\text{No}}, count_{invalid}).$$

La Scrutiny Authority produce infine una bacheca di scrutinio contenente il risultato aggregato e le informazioni pubbliche necessarie a verificare la corretta autorizzazione della fase di scrutinio:

$$TallyBoard = (ID_{Election}, Root_{final}, m, Tally, \{PublishedApproval_j\}, timestamp_{tally}, \Sigma_{SA}).$$

La Scrutiny Authority firma il risultato aggregato e lo collega alla radice finale del Bulletin Board. In questo modo il risultato pubblicato risulta riferito all'insieme cristallizzato delle schede cifrate, senza pubblicare una corrispondenza diretta tra indice del record e preferenza decifrata.

La fase di scrutinio si conclude quindi con un risultato aggregato collegato al Bulletin Board finale e alle autorizzazioni firmate dei nodi fiduciari.

1.17 Fase 10: pubblicazione del risultato e verificabilità

Terminato lo scrutinio, vengono pubblicate le informazioni necessarie a verificare la coerenza tra schede accettate, scrutinio e risultato finale.

La pubblicazione finale include:

- la $TallyBoard$ finale.
- il Bulletin Board finale BB_{final};
- la firma σ_{ER}^{Root};
- la dichiarazione firmata $PublishedVSStatement$ del Voting Server.

Ai fini della coerenza pubblica, il Voting Server pubblica una dichiarazione firmata relativa al numero totale di token definitivamente consumati, cioè al numero di entry in stato *committed* nel proprio registro interno $SpentTokensDB$.

$$VSStatement = (ID_{Election}, N_{committed}, timestamp_{VS}, Root_{final}).$$

dove:

$$N_{committed} = |\{e \in SpentTokensDB : e.status = committed\}|.$$

Il Voting Server firma la dichiarazione ed un osservatore esterno può verificare la firma del Voting Server tramite il certificato $Cert_{VS}$ e controllare la coerenza numerica tra il valore dichiarato e il numero di record presenti nel Bulletin Board finale:

Tale valore deve coincidere con il numero di record presenti nel Bulletin Board finale:

$$N_{committed} = |BB_{final}| = m.$$

Inoltre, dopo lo scrutinio deve valere:

$$m = count_{\text{Sì}} + count_{\text{No}} + count_{invalid}.$$

Questa verifica consente di controllare la coerenza numerica tra token consumati, schede pubblicate e schede conteggiate.

1.17.1 Verificabilità individuale

L'elettore utilizza $Receipt_U$ per verificare che la propria scheda cifrata sia inclusa nel Bulletin Board finale. In particolare, controlla:

1. che la firma del Registro Elettorale sulla ricevuta sia valida;
2. che la foglia $Leaf_i$ contenuta nella ricevuta sia presente nella struttura pubblicata;
3. che il Merkle path definitivo associato a $Leaf_i$ ricostruisca correttamente $Root_{final}$;
4. che $Root_{final}$ sia firmata dal Registro Elettorale.
5. verifica che $BallotCommit_U$ contenuto nella ricevuta coincida con il valore calcolato localmente dal Voting Client durante la preparazione della scheda;

6. verifica che il $BoardRecord_i$ incluso nel Bulletin Board contenga lo stesso $BallotCommit_U$ presente nella ricevuta.

L'uso del Merkle Tree rende efficiente questa operazione: l'elettore verifica l'inclusione tramite un cammino di autenticazione di dimensione logaritmica rispetto al numero di schede pubblicate. Questo gli permette di non dover verificare l'intera Bulletin Board. Questa verifica permette all'elettore di controllare l'inclusione della propria scheda cifrata, senza rivelare la preferenza espressa.

Verificabilità universale limitata. Nel protocollo proposto la verificabilità universale è intesa come verificabilità pubblica di coerenza, non come prova crittografica completa della corretta decifratura di ogni singola scheda.

Un osservatore esterno può verificare che il Bulletin Board finale sia coerente con la radice Merkle pubblicata, che la radice sia firmata dal Registro Elettorale, che i record inclusi non siano stati modificati dopo la chiusura delle urne, che le autorizzazioni dei nodi della Scrutiny Authority siano valide e che il risultato aggregato pubblicato rispetti la relazione:

$$m = count_{\text{Sì}} + count_{\text{No}} + count_{\text{invalid}}.$$

Questa forma di verifica collega pubblicamente il risultato finale all'insieme cristallizzato delle schede cifrate registrate. Tuttavia, un osservatore esterno non può verificare autonomamente che ciascun ciphertext sia stato decifrato correttamente, poiché non vengono usate prove pubbliche di corretta decifratura. La correttezza della fase di decifratura e conteggio rimane quindi affidata alla procedura congiunta della Scrutiny Authority distribuita. Per questo motivo, la proprietà viene considerata soddisfatta solo in forma parziale, sotto l'assunzione che la Scrutiny Authority operi correttamente e che non vi sia collusione sufficiente a manipolare lo scrutinio.

Algoritmi e Protocolli per la Sicurezza

Project Work

WP3: Analisi di sicurezza

Analisi critica del protocollo di voto elettronico proposto nel WP2

Autori: Gennaro Foschillo
Francesco Vittorio Allocca

A.A. 2025–2026

Indice

1 Analisi di sicurezza **1**

1.1 Obiettivo del Work Package . . . 1
1.2 Metodo di analisi . . . 1
1.3 Assunzioni di sicurezza considerate . . . 1
1.4 Criticità considerate . . . 2
1.5 Classificazione sintetica delle proprietà . . . 3

2 Analisi per proprietà **5**

2.1 Autenticità dell'elettore e del voto . . . 5
2.2 Autorizzazione e idoneità . . . 5
2.3 Unicità del voto . . . 6
2.4 Segretezza della preferenza . . . 6
2.5 Pseudoanonimato e separazione delle informazioni . . . 7
2.6 Integrità dei messaggi e del registro . . . 7
2.7 Verificabilità individuale . . . 8
2.8 Verificabilità universale . . . 8
2.9 Disponibilità . . . 8
2.10 Resistenza al replay . . . 9
2.11 Coercizione e voto di scambio . . . 9
2.12 Efficienza . . . 10

3 Analisi rispetto agli avversari del modello **10**

3.1 Elettore malevolo . . . 10
3.2 Voting Server malevolo . . . 10
3.3 Registro Elettorale malevolo . . . 11
3.4 Government Authority malevola o compromessa . . . 11
3.5 Scrutiny Authority collusa o compromessa . . . 11
3.6 Voting Client compromesso e phishing . . . 12
3.7 Avversario di rete . . . 12

4 Analisi dei compromessi progettuali **13**

5 Rischi residui **14**

6 Possibili estensioni future **14**

7 Conclusione dell'analisi **15**

1 Analisi di sicurezza

1.1 Obiettivo del Work Package

Questo Work Package analizza la sicurezza della soluzione proposta nel WP2 rispetto al modello definito nel WP1. L'obiettivo è valutare quali proprietà vengono soddisfatte e in quale misura, elencando eventuali rischi residui derivanti dal protocollo descritto nel WP2.

L'analisi considera il caso di referendum binario descritto nei WP precedenti. Le proprietà analizzate sono autenticità, autorizzazione, unicità del voto, segretezza della preferenza, pseudoanonimato, integrità, verificabilità individuale, verificabilità universale, resistenza al replay, disponibilità, coercizione ed efficienza.

1.2 Metodo di analisi

L'analisi viene svolta confrontando, per ciascuna proprietà:

- l'obiettivo dichiarato nel modello del WP1;
- i meccanismi effettivamente introdotti nel protocollo del WP2;
- le assunzioni necessarie affinché tali meccanismi siano efficaci;
- gli scenari di attacco mitigati dal protocollo;
- i limiti residui non eliminati dalla soluzione proposta.

L'analisi sottolinea le eventuali debolezze o criticità del sistema , verificando se esistono eventuali modifiche ovvie applicabili all'interno del WP2 che migliorino una proprietà senza peggiornarne altre. Quando una modifica produce benefici ma introduce costi, complessità o peggioramenti su altre proprietà, viene trattata come possibile estensione o come compromesso non adottato.

Si assume che le primitive crittografiche utilizzate siano implementate correttamente e che siano sicure rispetto ad avversari computazionalmente limitati. In particolare, l'analisi si basa su firme digitali non falsificabili, funzioni hash resistenti alle collisioni, cifratura asimmetrica probabilistica per la scheda, canali sicuri autenticati e strutture Merkle-based per la verifica di inclusione.

1.3 Assunzioni di sicurezza considerate

Assunzione	Descrizione
Primitive crittografiche sicure	Firme digitali, hash, cifratura probabilistica, MAC e canali sicuri sono considerati sicuri contro avversari efficienti. Una rottura di tali primitive invaliderebbe le garanzie associate.

Assunzione	Descrizione
Corretta gestione delle chiavi	Le chiavi private delle entità infrastrutturali e dell'elettore devono rimanere segrete. La compromissione di una chiave privata consente impersonificazione o accesso improprio alla fase corrispondente.
PKI e CRL disponibili	Certificati e liste di revoca pubblicati dalla Government Authority devono essere recuperabili, firmati e aggiornati. Le componenti devono rifiutare certificati scaduti, non coerenti con il ruolo o revocati.
Voting Client non pienamente fidato	Il client è eseguito sul dispositivo dell'elettore e può essere compromesso. Il protocollo può rilevare alcune sostituzioni tramite commitment e ricevuta, ma non può garantire pienamente che la preferenza visualizzata, quella cifrata e quella intesa dall'utente coincidano.
Separazione delle responsabilità	La sicurezza dipende dal fatto che le entità non colludano tutte tra loro. La separazione GA–VS–ER–SA riduce il rischio di associazione identità-voto, ma non elimina il rischio in caso di collusione estesa.
Scrutiny Authority distribuita e HSM	Almeno k nodi della Scrutiny Authority devono autorizzare lo scrutinio finale. La chiave privata di scrutinio sk_{SA} è custodita come chiave non esportabile all'interno di un HSM. Si assume che generi correttamente la coppia di chiavi
Sincronizzazione temporale ragionevole	Timestamp, finestre di validità, CRL e chiusura delle urne richiedono una sincronizzazione sufficiente tra le entità. Errori temporali possono causare rifiuti legittimi o accettazioni non desiderate.
Disponibilità minima dell'infrastruttura	La crittografia non impedisce attacchi di blocco, saturazione o indisponibilità. Si assume che GA, VS, ER, punti di pubblicazione e canali abbiano misure operative minime di disponibilità.

1.4 Criticità considerate

Prima di analizzare le singole proprietà, è utile isolare alcuni punti del protocollo che richiedono un'assunzione esplicita o che possono generare rischi residui.

Veridicità dello stato dei token spesi. Il registro $SpentTokensDB$ è interno al Voting Server. Un osservatore esterno non può sapere direttamente se il Voting Server stia dichiarando correttamente quali token siano in stato *pending* o *committed*. La verifica pubblica può controllare la coerenza tra i record pubblicati nel Bulletin Board, le ricevute firmate dal Registro Elettorale, la radice finale e il numero di schede scrutinate. Tuttavia, la correttezza completa dello stato interno del Voting Server resta un'assunzione operativa. Se il Voting Server è disonesto, può rifiutare richieste legittime, bloccare token o non inoltrare schede; tali comportamenti sono rilevabili solo in parte tramite assenza di ricevuta e incoerenze tra record attesi e record pubblicati.

Identità dell'entità nella richiesta di certificazione. La richiesta di certificazione

non può basarsi solo su un'identità dichiarata dall'entità richiedente. La GA deve verificare l'identità o il ruolo dell'entità prima di emettere il certificato, e deve richiedere anche una prova di possesso della chiave privata associata alla chiave pubblica da certificare. In caso contrario, un attaccante potrebbe chiedere un certificato per una chiave pubblica controllata da lui fingendosi Voting Server, Registro Elettorale o nodo della Scrutiny Authority. Questa criticità riguarda la fase di accreditamento e certificazione, non la cifratura del voto in sé.

Emissione concorrente dei token. Il vincolo di un solo token per la coppia $(ID_U, ID_{Election})$ richiede che l'aggiornamento di $IssuedTokensDB$ sia atomico. Se due richieste parallele dello stesso elettore vengono elaborate senza blocco o transazione, la GA potrebbe emettere due token validi prima di aggiornare lo stato. Per questo, l'analisi assume che la GA registri immediatamente lo stato di emissione del token.

Possibilità di risalire al voto di un elettore. Dal solo Bulletin Board pubblico non dovrebbe essere possibile risalire al voto di un elettore, perché i record pubblici contengono schede cifrate e non identità reali o token. Il rischio nasce dalla combinazione di più fonti: associazione identità-token nella GA, stato dei token nel VS, ordine o timestamp dei record nel Registro, log di rete e informazioni disponibili alla Scrutiny Authority durante lo scrutinio. Quindi il protocollo offre pseudoanonimato sotto separazione delle responsabilità, ma non anonimato assoluto contro collusione o compromissione estesa.

Dimostrabilità del voto da parte dell'elettore. La ricevuta del protocollo non contiene la preferenza e quindi non dovrebbe essere, da sola, una prova trasferibile del voto. Rimangono però prove esterne: screenshot, registrazioni dello schermo, osservazione fisica durante il voto, client controllato da terzi o conservazione dei dati locali usati per generare la scheda cifrata. In particolare, se il client conserva o rende esportabili i valori casuali usati nella cifratura probabilistica, l'elettore potrebbe tentare di dimostrare che una certa scheda cifrata corrisponde a una certa preferenza. La soluzione riduce le prove generate dal protocollo, ma non elimina la coercizione nel voto remoto.

HSM come root of trust dello scrutinio. Nel protocollo l'HSM è una componente fidata centrale. La sicurezza della fase di scrutinio dipende dal fatto che l'HSM applichi correttamente la politica di autorizzazione, accettando l'uso di sk_{SA} solo in presenza di almeno k approvazioni valide provenienti da nodi distinti della Scrutiny Authority.

1.5 Classificazione sintetica delle proprietà

La tabella seguente riassume il livello con cui il protocollo supporta le proprietà dichiarate nel modello. La classificazione distingue tra proprietà supportate direttamente, proprietà mitigate sotto assunzioni e proprietà non pienamente garantite.

Proprietà	**Livello**	**Meccanismi principali**	**Limite principale**
Autenticità	Supportata sotto assunzioni	Autenticazione GA, certificati, token firmato, binding con pk_U, firma σ_U^{Ballot}, verifica CRL	Dipende dalla correttezza della GA e dalla protezione di sk_U.

Proprietà	**Livello**	**Meccanismi principali**	**Limite principale**
Autorizzazione	Supportata sotto assunzioni	$EligibleUsersDB$, $IssuedTokensDB$, $Permit_U$, firma σ_{GA}^{Permit}	La GA rimane una componente fidata per l'idoneità.
Unicità	Supportata sotto assunzioni	Token unico, vincolo di emissione per $(ID_U, ID_{Election})$, $SpentTokensDB$, stati *pending*/*committed*	Dipende dall'atomicità dei controlli sul VS e dalla gestione delle anomalie *pending*.
Segretezza	Mitigata	Cifratura probabilistica con pk_{SA}, voto opaco per VS ed ER, scrutinio post-chiusura, HSM con chiave sk_{SA} non esportabile	Non resiste a client compromesso, HSM compromesso, uso improprio di sk_{SA} o collusione.
Pseudoanonimato	Mitigato	Separazione identità-token-voto, token non pubblicato, assenza della preferenza in chiaro durante raccolta e registrazione	Rimane possibile correlazione tramite log, timing, ordine dei record o collusione.
Integrità	Supportata con rilevabilità	Firme digitali, hash, commitment, Merkle Tree, ricevute firmate, $Root_{final}$	Alcune alterazioni vengono rese rilevabili, non sempre preventivamente impedite.
Verificabilità individuale	Supportata in forma limitata	$Receipt_U$, $BallotCommit_U$, $Leaf_i$, $MerklePath_i^{final}$, $Root_{final}$	Verifica l'inclusione della scheda cifrata, non prova da sola che il client abbia cifrato la preferenza voluta.
Verificabilità universale	Parziale	BB_{final}, $Root_{final}$, $TallyBoard$, firme ER/SA, controllo numerico	Non consente a un osservatore esterno di verificare autonomamente ogni decifratura effettuata dall'HSM.
Disponibilità	Non garantita crittograficamente	Hard-fail CRL, canali sicuri, separazione funzionale, controlli efficienti	DoS, blocchi di rete e indisponibilità dei servizi restano rischi residui.

Proprietà	Livello	Meccanismi principali	Limite principale
Efficienza	Considerata progettualmente	Referendum binario, Merkle path logaritmico, canale simmetrico di sessione, assenza di registro pubblico distribuito complesso	La misura quantitativa dei costi resta demandata al WP4.

2 Analisi per proprietà

2.1 Autenticità dell'elettore e del voto

L'autenticità dell'elettore è supportata dalla fase di autenticazione presso la Government Authority. Il modulo IP verifica l'identità reale dell'utente, mentre il modulo EV controlla la presenza dell'utente in $EligibleUsersDB$. Il token rilasciato dalla GA è firmato tramite sk_{GA}, quindi il Voting Server può verificare che l'autorizzazione provenga dall'autorità prevista.

L'autenticità della busta di voto è rafforzata dal binding tra $Permit_U$ e pk_U. Il Voting Client firma la busta tramite sk_U, e il Voting Server verifica la firma usando la chiave pubblica contenuta nel permesso firmato dalla GA. In questo modo, un avversario che ottiene solo il token, ma non sk_U, non dovrebbe poter produrre una busta valida.

Nella fase di certificazione delle entità infrastrutturali, la GA deve inoltre verificare che il richiedente sia davvero l'entità dichiarata e che possieda la chiave privata corrispondente alla chiave pubblica da certificare. Senza questa verifica iniziale, un certificato firmato dalla GA potrebbe legittimare una chiave controllata da un attaccante.

Il limite principale è che l'autenticità dipende dalla protezione della chiave privata dell'elettore e dalla correttezza della GA. Se il dispositivo dell'elettore è compromesso o se sk_U viene sottratta, l'attaccante può votare come l'utente. Se la GA è disonesta, può emettere token in modo improprio o alterare la procedura di idoneità.

2.2 Autorizzazione e idoneità

L'autorizzazione è separata dall'autenticazione. Un utente può essere autenticato correttamente, ma riceve il token solo se risulta idoneo per $ID_{Election}$. Questa distinzione è coerente con il modello del WP1, perché evita di confondere identità digitale e diritto di partecipazione alla specifica consultazione.

Il registro $IssuedTokensDB$ impedisce, sotto assunzioni di correttezza della GA, l'emissione di più token allo stesso elettore per la stessa elezione. Questa garanzia richiede che controllo ed aggiornamento dello stato siano atomici: in presenza di richieste parallele, la GA deve segnare immediatamente l'emissione o bloccare la coppia $(ID_U, ID_{Election})$ prima di restituire un nuovo token. Questa scelta tutela l'unicità già nella fase di autorizzazione, prima ancora della raccolta del voto.

Il database dei token emessi è però sensibile. Se conserva in chiaro l'associazione tra identità reale e token e stato di esso, una sua compromissione non rivela direttamente la preferenza, ma fornisce un elemento forte per correlazioni successive con log del VS, metadati di rete o informazioni di scrutinio. Questo rischio è mitigato salvando i token cifrati precedentemente con la chiave pubblica dell'utente.

Il rischio residuo è che $IssuedTokensDB$ è un registro interno della GA. Un osservatore esterno non può verificare direttamente che la GA non abbia emesso token impropri, a meno che non siano previste procedure amministrative o verifiche esterne. Nel protocollo proposto, questa rimane un'assunzione di fiducia sulla Government Authority.

2.3 Unicità del voto

L'unicità è supportata da due livelli. Il primo livello è lato GA: per ogni coppia $(ID_U, ID_{Election})$ deve essere emesso al più un token. Il secondo livello è lato VS: il token non può essere riutilizzato perché il Voting Server memorizza in $SpentTokensDB$ un digest non invertibile derivato da $SerialNumber_{Token}$ e da un hash deterministico legato a K_{Master_Server}.

La distinzione tra stato *pending* e stato *committed* è rilevante. Lo stato *pending* blocca subito riusi concorrenti del token dopo la verifica positiva del Voting Server. Lo stato *committed* viene assegnato solo dopo la ricezione di una ricevuta firmata dal Registro Elettorale. Questa scelta evita che un token venga considerato definitivamente speso prima che esista una prova firmata di presa in carico della scheda.

Un limite importante è che $SpentTokensDB$ non è pubblico, almeno fino alla chiusura delle urne. Il sistema può pubblicare conteggi, ricevute e record finali, ma un verificatore esterno non vede direttamente lo stato interno usato dal VS per decidere se un token è speso. La garanzia di unicità dipende quindi dalla correttezza del VS e dalla possibilità di rilevare anomalie tramite ricevute e coerenza tra token in stato committed e schede effettivamente presenti nel Bulletin Board.

Il rischio residuo riguarda le anomalie operative. Se il Registro Elettorale non risponde, se la ricevuta non arriva o se il canale tra VS ed ER fallisce, il token può rimanere in stato *pending*. Sbloccare automaticamente il token migliorerebbe la disponibilità, ma aprirebbe il rischio di doppio voto in presenza di ritardi o risposte duplicate. Per questo motivo, nel protocollo fissato, una entry *pending* persistente deve essere trattata come anomalia, non come semplice rollback automatico.

2.4 Segretezza della preferenza

La segretezza della preferenza è supportata dalla cifratura probabilistica della scheda tramite pk_{SA}. Il Voting Server e il Registro Elettorale ricevono e pubblicano $EncVoto_{SA}$, non il voto in chiaro. L'uso di cifratura probabilistica è necessario soprattutto nel referendum binario: se la cifratura fosse deterministica, due voti uguali produrrebbero lo stesso ciphertext e un osservatore potrebbe riconoscere pattern tra schede pubblicate.

La segretezza vale però solo sotto ipotesi precise. Il Voting Client vede la preferenza prima della cifratura; quindi un client compromesso può leggerla o modificarla. La

Scrutiny Authority, durante lo scrutinio, accede alle preferenze decifrate, dunque se la SA conservasse una mappa tra indice del record e preferenza, o colludesse con ER e VS, il rischio di ricostruzione del voto individuale aumenta. La soluzione mitiga il problema evitando di pubblicare la corrispondenza della i-sima scheda con il voto associato, pubblicando solo il risultato aggregato.

2.5 Pseudoanonimato e separazione delle informazioni

Il protocollo separa le informazioni trattate dalle diverse entità. La GA conosce l'identità reale dell'elettore e rilascia un token, ma non riceve la scheda cifrata nella fase di voto. Il Voting Server verifica il token e la prova di possesso, ma non dovrebbe conoscere l'identità reale dell'elettore né la preferenza. Il Registro Elettorale pubblica schede cifrate e commitment, ma non token o identità. La Scrutiny Authority opera sul Bulletin Board finale, non sull'elenco degli elettori.

Questa separazione supporta lo pseudoanonimato, ma non garantisce anonimato assoluto. Un avversario che combini log di rete, timestamp, ordine dei record, informazioni del VS e informazioni della GA può tentare correlazioni. Il rischio cresce se più entità colludono o se i log operativi sono troppo dettagliati.

In questa sezione è quindi corretto parlare di pseudoanonimato mitigato, non di anonimato forte. Il rischio residuo di correlazione viene quindi accettato come conseguenza del protocollo scelto e del vincolo di efficienza.

2.6 Integrità dei messaggi e del registro

L'integrità dei messaggi è supportata da firme digitali, certificati, MAC del canale sicuro e controlli di freschezza. La sola cifratura non viene considerata sufficiente a garantire integrità; per questo il protocollo usa firme sugli oggetti persistenti e canali autenticati per gli scambi di sessione.

Il Registro Elettorale fornisce integrità e rilevabilità delle manomissioni tramite una struttura append-only e tramite Merkle Tree. Ogni record produce una foglia (Leaf_i), e la radice finale (Root_{final}) viene firmata dal Registro. Una modifica a un record pubblicato altererebbe la foglia corrispondente e quindi la radice finale, rendendo rilevabile la manipolazione.

Nel protocollo, la busta di voto inviata dall'elettore è firmata con la chiave privata associata al token di voto. Se la firma dell'elettore copre $EncVoto_{SA}$, $nonce_{ballot}$, $VotingToken_U$ e

$BallotCommit_U$, il Voting Server non può sostituire la scheda producendo una nuova busta valida senza falsificare la firma. In questo caso, una sostituzione del voto cifrato renderebbe la firma non valida oppure produrrebbe un disallineamento rilevabile tra la scheda preparata localmente, il commitment, la ricevuta e il record pubblicato.

Questo non elimina però tutti i comportamenti malevoli possibili da parte del Voting Server. Il server può ancora rifiutare una scheda valida, non inoltrarla al Registro Elettorale, bloccare un token in stato *pending* oppure ritardare la produzione della ricevuta. Tali comportamenti non sono impediti crittograficamente, ma possono essere rilevati solo parzialmente tramite assenza di ricevuta e controlli di coerenza tra token

consumati, record pubblicati e risultato finale.

2.7 Verificabilità individuale

La verificabilità individuale permette all'elettore di controllare che la propria scheda cifrata sia stata inclusa nel Bulletin Board finale. L'elettore verifica la firma del Registro sulla ricevuta, la presenza della foglia nella struttura pubblicata, il Merkle path definitivo e la firma sulla radice finale. Verifica inoltre che $BallotCommit_U$ nella ricevuta e nel record coincida con il valore calcolato localmente.

Questa proprietà permette all'elettore di controllare che la scheda cifrata registrata sia quella associata al commitment prodotto dal client. Non dimostra però, da sola, che la preferenza cifrata corrisponda alla scelta che l'elettore intendeva esprimere. Se il client mostra "Sì" ma cifra "No", il protocollo base non fornisce una prova indipendente capace di rilevare sempre l'errore.

La ricevuta non contiene la preferenza in chiaro e non include una prova pubblica dell'opzione votata. Questo riduce il rischio che venga usata come prova trasferibile in scenari di coercizione o compravendita del voto.

2.8 Verificabilità universale

La verificabilità universale del protocollo è una verificabilità pubblica di coerenza. Un osservatore può controllare che i record del Bulletin Board siano coerenti con $Root_{final}$, che la radice sia firmata dal Registro Elettorale, che il numero di record coincida con $N_{committed}$ dichiarato e firmato, che vi siano almeno k autorizzazioni valide della Scrutiny Authority e che il risultato aggregato rispetti la relazione:

$$m = count_{\text{Sì}} + count_{\text{No}} + count_{invalid}.$$

Questa verifica è utile perché collega il risultato all'insieme cristallizzato dei record pubblicati. Tuttavia, non permette a chiunque di verificare autonomamente che ogni ciphertext sia stato decifrato correttamente e conteggiato nel modo giusto. Mancano infatti prove pubbliche di corretta decifratura per ogni singola scheda. Questa scelta mantiene il protocollo più semplice, ma limita la verificabilità universale.

Per questo motivo, nel documento la verificabilità universale è dichiarata come proprietà rispettata parzialmente. Il sistema offre trasparenza pubblica su record, firme, radice Merkle, autorizzazioni e coerenza numerica, ma la correttezza interna della decifratura rimane affidata alla procedura congiunta della Scrutiny Authority.

2.9 Disponibilità

La disponibilità non è garantita direttamente dalle primitive crittografiche. Un avversario può bloccare il Voting Client, saturare il Voting Server, rendere irraggiungibile il punto di pubblicazione della CRL, ritardare la risposta del Registro Elettorale o impedire la cooperazione di almeno k nodi della Scrutiny Authority.

Il protocollo contiene alcune scelte che riducono il carico e quindi aiutano indirettamente la disponibilità. Il Voting Server esegue prima controlli economici e solo dopo verifiche crittografiche più costose. Le entità possono mantenere in cache oggetti firmati come $Params_{Election}$ e $CRL_{Election}$, controllandone versione e validità. La verifica Merkle ha costo logaritmico per l'elettore.

Rimane però un compromesso: il comportamento hard-fail sulla CRL migliora la sicurezza contro certificati revocati, ma peggiora la disponibilità quando la CRL non è recuperabile o verificabile. Questa scelta è dichiarata come trade-off consapevole.

2.10 Resistenza al replay

Il protocollo mitiga i replay tramite nonce, timestamp, finestre temporali e stato dei token. Un token già usato produce lo stesso riferimento in $SpentTokensDB$ e viene rifiutato se risulta in stato *pending* o *committed*. La busta di voto contiene $nonce_{ballot}$, e il canale sicuro richiede freschezza della sessione.

Il rischio residuo non è tanto la riaccettazione dello stesso token, quanto la gestione di replay o ritardi in condizioni di rete anomale. Ad esempio, se un messaggio viene ritardato mentre il token è in stato *pending*, il sistema può produrre una situazione di blocco operativo. Questa è una criticità di robustezza e disponibilità, non una rottura diretta dell'unicità.

2.11 Coercizione e voto di scambio

Il protocollo non elimina totalmente la coercizione. Il voto avviene in un ambiente remoto non presidiato, quindi un coercitore può osservare fisicamente l'elettore, imporre l'uso di un client controllato o richiedere prove esterne come registrazioni e screenshot.

La ricevuta del protocollo non contiene la preferenza e non pubblica la corrispondenza tra record e voto decifrato. Questa scelta riduce la produzione di prove trasferibili generate direttamente dal sistema. Tuttavia, non impedisce prove esterne prodotte dall'utente o dal dispositivo. In particolare, screenshot, registrazioni dello schermo, osservazione fisica, malware sul client o conservazione della casualità usata nella cifratura possono permettere all'elettore di dimostrare il voto a un terzo. Per questo la soluzione non deve essere presentata come resistente alla coercizione in senso forte.

La firma dell'elettore sulla busta di voto non deve essere interpretata come non ripudio della preferenza. Nel protocollo, tale firma serve a dimostrare il possesso della chiave associata al token e a proteggere l'integrità della busta inviata, non a produrre una prova pubblicamente trasferibile del contenuto del voto. Una prova trasferibile della preferenza sarebbe anzi indesiderabile, perché aumenterebbe il rischio di coercizione e compravendita del voto.

L'assenza di sostituzione del voto semplifica unicità e scrutinio, ma rende il sistema meno resistente alla coercizione rispetto a modelli che consentono di votare di nuovo e considerare valido solo l'ultimo voto. Tale meccanismo non è stato adottato perché avrebbe aumentato complessità, gestione dello stato, rischio di contestazioni e carico sul protocollo.

2.12 Efficienza

L'efficienza è considerata come vincolo progettuale già nel modello e nella soluzione. Le scelte principali sono:

- referendum binario, che semplifica formato della scheda e scrutinio;
- uso della crittografia asimmetrica solo per chiavi, firme, token e cifratura della preferenza;
- uso di chiave di sessione simmetrica per proteggere i messaggi successivi al canale sicuro;
- Merkle Tree per prove di inclusione di dimensione logaritmica;
- assenza di un registro pubblico distribuito complesso e di protocolli di decifratura più avanzati come meccanismi principali;
- controlli del Voting Server ordinati dal meno costoso al più costoso;
- pubblicazione aggregata del risultato, evitando di pubblicare grandi prove crittografiche non previste dal protocollo base.

La valutazione quantitativa di tempi, dimensioni dei messaggi e costi computazionali deve essere svolta nel WP4.

3 Analisi rispetto agli avversari del modello

3.1 Elettore malevolo

Un elettore malevolo può tentare di votare più volte, riutilizzare il token, inviare schede malformate, ripudiare il voto o produrre prove da mostrare a terzi. Il riuso del token viene mitigato da $SpentTokensDB$, mentre le schede malformate possono essere scartate in fase di verifica o marcate come invalide in fase di scrutinio.

Un elettore autorizzato può comunque consumare il proprio token con una scheda non valida. Questo non altera voti altrui, ma può aumentare $count_{invalid}$. Il protocollo non impedisce a un elettore di produrre prove esterne del proprio voto tramite strumenti fuori dal protocollo.

3.2 Voting Server malevolo

Un Voting Server malevolo può tentare di accettare token non validi, riutilizzare token, sopprimere schede valide, sostituire schede cifrate o non inoltrare correttamente i record al Registro Elettorale. Le firme sui token e la verifica di σ_U^{Ballot} mitigano l'accettazione di schede non autorizzate. $SpentTokensDB$ mitiga il riuso del token. Il commitment e la ricevuta del Registro rendono rilevabile dall'elettore una sostituzione della scheda.

Il protocollo non impedisce completamente la soppressione di una scheda prima della registrazione. Se il Voting Server rifiuta o scarta una busta valida, l'elettore può accorgersi

dell'assenza di ricevuta, ma il sistema non garantisce da solo la consegna forzata al Registro. Questo è un rischio residuo di disponibilità e correttezza operativa.

3.3 Registro Elettorale malevolo

Un Registro Elettorale malevolo può tentare di non pubblicare un record, alterare record già pubblicati, produrre ricevute scorrette o pubblicare una radice finale non coerente. La firma dei record, il Merkle Tree, la ricevuta firmata e la pubblicazione di $Root_{final}$ rendono rilevabili molte manipolazioni.

Se il Registro non inserisce una scheda, l'elettore non ottiene una ricevuta valida o non riesce a trovare la propria foglia nella struttura finale. Se altera un record dopo la pubblicazione, la prova Merkle o la radice finale non risultano coerenti.

Rimane il rischio di collusione tra Registro e Voting Server. Se entrambi coordinano rifiuti o omissioni prima della produzione di ricevute, la crittografia rende più facile rilevare anomalie, ma non garantisce piena disponibilità del voto.

3.4 Government Authority malevola o compromessa

La Government Authority è una componente fortemente fidata. Può emettere certificati, revocare entità, pubblicare parametri elettorali, autenticare utenti e rilasciare token. Una GA malevola potrebbe emettere token non autorizzati, negare token a elettori legittimi, pubblicare parametri scorretti o certificare entità non affidabili.

Le firme della GA garantiscono autenticità degli oggetti pubblicati, ma non garantiscono che la GA abbia operato correttamente dal punto di vista amministrativo. Nel protocollo base, questa è una fiducia residua dichiarata.

3.5 Scrutiny Authority collusa o compromessa

La Scrutiny Authority è distribuita per evitare che un singolo nodo fiduciario possa autorizzare da solo lo scrutinio finale. Almeno k nodi devono produrre approvazioni firmate, e tali approvazioni vengono verificate prima di consentire l'uso della chiave privata di scrutinio. Nel protocollo aggiornato, però, i nodi SA_i non possiedono quote crittografiche di sk_{SA} e non producono decifrature parziali. La chiave privata di scrutinio sk_{SA} è custodita come chiave non esportabile all'interno dell'HSM associato alla Scrutiny Authority. L'HSM esegue internamente la decifratura delle schede solo dopo aver verificato la presenza di almeno k approvazioni valide provenienti da nodi distinti. Questa scelta riduce il rischio che un singolo nodo software possa usare autonomamente sk_{SA}, ma non equivale a una threshold decryption matematica. Un attaccante che comprometta meno di k nodi della Scrutiny Authority non dovrebbe poter autorizzare lo scrutinio da solo. Tuttavia, se almeno k nodi colludono, essi possono autorizzare l'uso dell'HSM secondo la politica prevista. Inoltre, se l'HSM, il suo firmware o la sua configurazione fossero compromessi, la protezione di sk_{SA} potrebbe essere aggirata. In tal caso, la segretezza delle schede e l'integrità della fase di scrutinio risulterebbero compromesse.

3.6 Voting Client compromesso e phishing

Il Voting Client compromesso è uno degli avversari più rilevanti. Può mostrare opzioni corrette ma cifrare una preferenza diversa, conservare la preferenza, inviare metadati a terzi, rubare sk_U, alterare il contenuto della busta di voto o impedire all'utente di completare la verifica finale.

Caso particolare è il phishing, Un attaccante può tentare di indurre l'elettore a usare un'applicazione falsa, una pagina web contraffatta o un link malevolo che imita il Voting Client ufficiale. Il rischio è maggiore perché l'attacco non richiede necessariamente di rompere le primitive crittografiche: è sufficiente convincere l'elettore a interagire con un client non autentico.

Nel protocollo questo rischio è mitigato assumendo che il Voting Client venga ottenuto esclusivamente da una fonte ufficiale e verificabile. In particolare, il client deve essere distribuito tramite un canale riconosciuto dall'autorità elettorale e deve essere firmato o comunque autenticabile rispetto alla Government Authority. L'elettore non dovrebbe usare link ricevuti tramite canali non affidabili, mentre il client ufficiale deve verificare $Params_{Election}$, i certificati delle entità coinvolte, la $CRL_{Election}$ e l'autenticità del Voting Server prima dell'invio della scheda.

Queste misure riducono il rischio di phishing e di sostituzione del client, ma non lo eliminano completamente. Se l'elettore installa volontariamente un'applicazione falsa, accede a una pagina contraffatta o utilizza un dispositivo già compromesso, il protocollo non può garantire che la preferenza visualizzata, quella cifrata e quella effettivamente inviata coincidano. Il rischio residuo deve quindi essere dichiarato chiaramente: la sicurezza del voto remoto dipende anche dalla capacità dell'utente di usare il client ufficiale e dall'integrità del dispositivo su cui il client viene eseguito.

3.7 Avversario di rete

Un avversario di rete può intercettare, modificare, ritardare o ripetere messaggi. Il canale sicuro autenticato, il key exchange autenticato, i MAC, le firme, i nonce e i timestamp mitigano eavesdropping, manomissione e replay.

Rimangono possibili attacchi di disponibilità. Un avversario può bloccare il traffico, impedire il recupero di ($Params_{Election}$) o ($CRL_{Election}$), saturare il Voting Server o ritardare la consegna della ricevuta. La crittografia rileva o impedisce molte alterazioni, ma non garantisce la consegna dei messaggi.

Per quanto riguarda attacchi basati su ciphertext scelti o modificati, il protocollo mitiga il rischio assumendo l'uso di una cifratura asimmetrica probabilistica e non malleabile per la scheda. In particolare, la scheda cifrata viene trattata come oggetto opaco da Voting Server e Registro Elettorale, mentre la decifratura è eseguita solo dalla Scrutiny Authority dopo la chiusura delle urne.

Questa scelta riduce la superficie di attacco rispetto a scenari interattivi in cui un avversario può inviare ciphertext modificati e osservare risposte dettagliate dal destinatario. Il protocollo non prevede infatti che la Scrutiny Authority fornisca feedback in tempo reale sulla validità o sul contenuto delle singole schede cifrate. Eventuali ciphertext non

decifrabili o non conformi devono essere gestiti in modo uniforme durante lo scrutinio, evitando messaggi di errore dettagliati che possano trasformare la Scrutiny Authority in un oracolo di decifratura.

Questa mitigazione dipende però dalla corretta scelta e implementazione dello schema di cifratura e dalla gestione uniforme degli errori in fase di scrutinio. Pertanto, nel modello analizzato si applica una riduzione pratica del rischio di attacchi di tipo chosen-ciphertext sotto le assunzioni crittografiche e operative dichiarate.

4 Analisi dei compromessi progettuali

Il protocollo adotta compromessi espliciti. Tali compromessi non rappresentano errori, ma scelte motivate rispetto agli obiettivi della traccia, agli strumenti trattati nel corso e al vincolo di efficienza.

Scelta	**Motivazione**	**Rischio residuo**
Referendum binario	Riduce complessità della scheda e dello scrutinio.	Minore generalità rispetto a elezioni con preferenze multiple o liste complesse.
Nessuna sostituzione del voto	Mantiene semplice l'unicità del token e riduce ambiguità nello scrutinio.	L'elettore non può correggere un voto inviato e la resistenza alla coercizione è più debole.
Token non pubblicato sul Bulletin Board	Riduce la correlazione tra autorizzazione e scheda registrata.	La verifica pubblica dell'emissione corretta dei token resta limitata.
Pubblicazione del solo tally aggregato	Riduce il rischio di collegare record e preferenza decifrata.	Un osservatore esterno non verifica ogni decifratura individuale.
Custodia distribuita di sk_{SA}	Evita primitive avanzate non necessarie come soluzione principale.	Non offre una prova pubblica della corretta decifratura di ogni scheda.
Nessun registro distribuito complesso	Evita latenza, costi e complessità non proporzionati al progetto.	La trasparenza dipende dal Bulletin Board, dalle firme e dalla disponibilità dei dati pubblicati.
Nessun riordinamento crittografico delle schede	Mantiene il protocollo più semplice e più efficiente.	Resta rischio di correlazione tramite timing, ordine e metadati.
Hard-fail sulla CRL	Evita di accettare certificati revocati o stato di fiducia obsoleto.	Può interrompere operazioni legittime se la CRL non è raggiungibile.

5 Rischi residui

I principali rischi residui sono i seguenti:

- compromissione del Voting Client o del dispositivo dell'elettore;
- furto o uso improprio di sk_U, con conseguente voto non autorizzato dal punto di vista dell'utente;
- collusione tra più entità infrastrutturali, in particolare tra GA, VS, ER e SA;
- uso improprio o compromissione della chiave privata di scrutinio sk_{SA};
- compromissione dell'HSM, del suo firmware o della sua politica di autorizzazione, con conseguente uso improprio di sk_{SA};
- correlazione tramite log temporali, indirizzi di rete, ordine dei record o timestamp;
- soppressione di schede prima della produzione della ricevuta firmata;
- stati *pending* persistenti in $SpentTokensDB$, con blocco operativo del token;
- attacchi alla disponibilità di GA, VS, ER, CRL o canali di comunicazione;
- CRL non raggiungibile o non aggiornata, con conseguente hard-fail;
- assenza di piena resistenza alla coercizione nel voto remoto non presidiato;
- verificabilità universale limitata alla coerenza pubblica, senza prova autonoma di ogni decifratura.

Questi rischi non annullano l'utilità del protocollo, ma definiscono il perimetro reale delle garanzie offerte. La soluzione è difendibile se presentata come compromesso ragionevole e non come sistema di voto elettronico perfetto.

6 Possibili estensioni future

- **Rafforzamento della certificazione**: la fase di emissione dei certificati potrebbe includere in modo più esplicito challenge–response, prova di possesso della chiave privata e controlli più stringenti sul ruolo dell'entità richiedente.
- **Log firmati**: GA, VS ed ER potrebbero produrre log firmati degli eventi critici, senza pubblicare dati sensibili, per rendere più controllabili emissione dei token, stati *pending*/*committed* e pubblicazione dei record.
- **Threshold decryption crittografica**: una possibile estensione futura consisterebbe nel sostituire la custodia tramite HSM con una vera decifratura a soglia, in cui la chiave privata di scrutinio non esiste mai interamente in un singolo componente e i nodi producono contributi di decifratura parziale. Questa soluzione ridurrebbe la fiducia nell'HSM come root of trust, ma richiederebbe primitive e protocolli più avanzati rispetto alla soluzione base.

- **Procedure più rigorose per i token in stato *pending***: si potrebbero definire timeout, contestazioni e controlli manuali firmati per evitare sia il blocco permanente del token sia il rischio di doppio utilizzo.

Queste estensioni non modificano la natura del protocollo. Servono soprattutto a ridurre rischi operativi già individuati nell'analisi, senza trasformare la soluzione in un sistema basato su primitive avanzate non necessarie per il progetto base.

7 Conclusione dell'analisi

Il protocollo proposto nel WP2 realizza un compromesso ragionevole tra sicurezza, trasparenza ed efficienza. Le proprietà meglio supportate sono autenticità, autorizzazione, unicità sotto assunzioni, integrità dei record e verificabilità individuale dell'inclusione. La segretezza e lo pseudoanonimato sono mitigati dalla cifratura e dalla separazione delle responsabilità, ma non sono assoluti. La verificabilità universale è presente come controllo pubblico di coerenza tra Bulletin Board, radice Merkle, autorizzazioni di scrutinio e risultato aggregato, ma non come verifica crittografica completa di ogni decifratura. La principale debolezza tuttavia è che alcune garanzie dipendono dalla correttezza operativa e dalla non collusione delle entità infrastrutturali.

Algoritmi e Protocolli per la Sicurezza

WP4 E-vote Proof of Concept

Implementazione e prestazioni della PoC Python del protocollo di voto elettronico

Gruppo 16: Gennaro Foschillo, Francesco Vittorio Allocca

A.A. 2025–2026

Indice

1 Panoramica **3**

2 Architettura e ruolo delle entità **3**

2.1 Government Authority (GA) . . . 3

2.2 Voting Client (VC) . . . 3

2.3 Voting Server (VS) . . . 4

2.4 Electoral Register (ER) . . . 4

2.5 Scrutiny Authority (SA) . . . 4

2.6 Moduli trasversali . . . 4

3 Scelte tecnologiche e librerie **5**

3.1 Linguaggio e runtime . . . 5

3.2 Dipendenze esterne . . . 5

3.3 Primitive crittografiche . . . 5

3.4 Scelte architetturali rilevanti . . . 6

4 Analisi prestazioni da `benchmark_results.csv` **6**

4.1 Latenza end-to-end e throughput . . . 6

4.2 Tempi crittografici medi . . . 6

4.3 Costi totali rilevanti . . . 7

5 Dimensione credenziali e messaggi **7**

6 Latenza di verifica **8**

7 Misurazioni effettuate **8**

8 Flusso protocollo **9**

9 Limitazioni e funzionalità non implementate **9**

9.1 Infrastruttura . . . 9

9.2 Sicurezza e PKI . . . 9

9.3 Crittografia avanzata . . . 9

9.4 Operatività elettorale 10
9.5 Test e robustezza 10

10 Conclusione **10**

1 Panoramica

La PoC implementa un **referendum binario** (`YES` / `NO`) come sistema stand-alone in Python. Non c'è rete reale, database persistente né blockchain: tutte le entità sono **classi in memoria** che comunicano tramite chiamate dirette.

Il flusso copre l'intero ciclo elettorale: setup, autenticazione, emissione token, voto cifrato, registrazione su Bulletin Board, ricevuta, anti-riuso token, Merkle tree, verifica individuale, scrutinio e verifica universale simulata.

2 Architettura e ruolo delle entità

2.1 Government Authority (GA)

File: `government_authority/`

Facade che orchestra quattro sottomoduli:

Modulo	Ruolo
Identity Provider (IP)	Autentica l'utente interno tramite `authenticate_internal`.
Eligibility Verifier (EV)	Verifica che l'utente sia nella lista degli aventi diritto.
Certificate Authority (CA)	Emette certificati firmati, gestisce CRL e firma payload.
Voting Token Issuer (VTI)	Emette al massimo un token per $(user_id, election_id)$, firmato dalla GA e cifrato con chiave pubblica dell'elettore.

La GA pubblica anche il **manifesto elettorale** (`ElectionParameters`): opzioni, chiavi pubbliche, soglia scrutinio e seriali certificati.

2.2 Voting Client (VC)

File: `voting_client/voting_client.py`

Simula l'elettore:

1. genera una coppia RSA personale;
2. invia `AuthRequest` con `election_id`, chiave pubblica, nonce e timestamp;
3. riceve e decifra `EncToken` tramite cifratura ibrida RSA+Fernet;
4. crea `Ballot`: voto cifrato con chiave SA, commitment e firma elettore;
5. apre un canale sicuro simulato verso VS dopo verifica del certificato;

6. verifica `Receipt` e inclusione Merkle.

2.3 Voting Server (VS)

File: `voting_server/voting_server.py`

Punto di raccolta e controllo:

- verifica certificato ER, freschezza busta e nonce anti-replay;
- verifica token GA: firma, validità temporale e binding alla chiave elettore;
- applica anti-riuso token tramite `SpentTokensDB`: tag HMAC derivato da `K_Master_Server` e `serial_number`, senza salvare il seriale in chiaro;
- verifica commitment della scheda;
- firma e inoltra `Mboard` all'ER.

2.4 Electoral Register (ER)

File: `electoral_register/electoral_register.py`

Implementa il **Bulletin Board append-only**:

- ogni `BoardRecord` contiene un `Mboard` firmato dal VS, timestamp ER e hash del record precedente;
- emette `Receipt` firmata, contenente `leaf_hash`, `ballot_commit` e `encrypted_vote_hash`;
- a chiusura urne costruisce il **Merkle tree** e genera proof di inclusione.

2.5 Scrutiny Authority (SA)

File: `scrutiny_authority/scrutiny_authority.py`

Scrutinio simulato con **k-of-n**, di default 3 su 5 nodi:

- i nodi `SA_NODE` firmano `ScrutinyAuthorization` su $(election_id, root_final, record_count)$;
- dopo il quorum, la SA decifra i voti con la propria chiave privata e produce `TallyResult`;
- non è implementata threshold decryption matematica: il quorum è solo una fase di autorizzazione distribuita.

2.6 Moduli trasversali

Modulo	Ruolo
`core/models.py`	Dataclass del protocollo: `Certificate`, `Ballot`, `BoardRecord`, `Receipt`, ecc.
`crypto/crypto_utils.py`	Primitive crittografiche e canale sicuro simulato.
`crypto/certificates.py`	Verifica certificati e CRL.
`crypto/merkle.py`	Costruzione albero e proof.
`core/verification.py`	Verifica universale simulata.
`core/metrics.py`	Raccolta tempi e dimensioni per benchmark.

3 Scelte tecnologiche e librerie

3.1 Linguaggio e runtime

- Python 3.10+;
- strutture dati: `dataclass`, `dict`, `list`, `set` in memoria;
- serializzazione deterministica: JSON canonico con `sort_keys=True`.

3.2 Dipendenze esterne

```
cryptography>=42.0.0
```

Tutto il resto è **stdlib**: `json`, `hashlib`, `hmac`, `base64`, `os`, `time`, `statistics`, `csv`, `threading`, `uuid`.

3.3 Primitive crittografiche

Uso	Algoritmo
Chiavi asimmetriche	RSA 2048 bit, esponente 65537.
Firme digitali	RSA-PSS + SHA-256.
Cifratura scheda elettorale	RSA-OAEP + SHA-256, non deterministica.
Cifratura token, payload grande	Ibrida: Fernet + RSA wrap.
Canale VC–VS	Fernet, come simulazione TLS.
Commitment / hash catena / Merkle	SHA-256.
Lookup token spesi	HMAC-SHA256 con chiave derivata per elezione.
Merkle tree	SHA-256 su foglie JSON canoniche.

3.4 Scelte architetturali rilevanti

- certificati e CRL sono strutture dati firmate, non X.509;
- il voto in chiaro resta solo nel client; VS ed ER vedono solo ciphertext e commitment;
- la ricevuta non contiene la preferenza, solo hash e riferimenti crittografici;
- thread lock nel VTI per emissione token concorrente simulata.

4 Analisi prestazioni da `benchmark_results.csv`

Gli esperimenti sono stati eseguiti con 10, 100, 500 e 1000 elettori. Tutti i voti risultano completati con successo.

4.1 Latenza end-to-end e throughput

Elettori	Latenza media/voto	Throughput (voti/s)
10	42.2 ms	23.7
100	44.8 ms	22.3
500	56.0 ms	17.8
1000	50.1 ms	20.0

La latenza per voto resta nell'ordine dei 40–56 ms, con throughput stabile intorno ai 18–24 voti/s. Il calo a 500 elettori è probabilmente legato al carico di generazione chiavi RSA, poiché ogni elettore genera una coppia.

4.2 Tempi crittografici medi

Operazione	10 elettori	1000 elettori
Firma (`sign`)	0.65 ms	0.69 ms
Verifica firma	0.04 ms	0.06 ms
Cifratura voto	0.03 ms	0.05 ms
Decifratura voto, scrutinio	0.42 ms	0.54 ms
Generazione token	0.69 ms	0.93 ms
Verifica token	∼0.0006 ms	∼0.0005 ms
Registrazione board	0.016 ms	0.020 ms
Verifica Merkle path	0.005 ms	0.011 ms

Osservazioni.

- La generazione chiavi RSA domina il costo totale di setup: da 0.69 s per 10 elettori a 44.9 s per 1000 elettori, perché ogni `VotingClient` genera una coppia RSA e l'ambiente genera anche le chiavi di VS, ER, SA e 5 nodi.
- Lo scrutinio scala linearmente: 5.5 ms per 10 voti e 541 ms per 1000 voti, coerentemente con la decifratura RSA scheda per scheda.
- Il Merkle build resta trascurabile fino a 1000 record, circa 10 ms.

4.3 Costi totali rilevanti

Elettori	Key generation totale	Merkle build	Scrutinio totale
10	0.69 s	0.14 ms	5.5 ms
100	4.39 s	0.81 ms	47.6 ms
500	25.8 s	6.6 ms	247 ms
1000	44.9 s	9.8 ms	541 ms

5 Dimensione credenziali e messaggi

Dimensioni medie in byte, calcolate su JSON canonico, stabili rispetto al numero di elettori.

Messaggio	Dimensione media	Contenuto principale
AuthRequest	603 byte	`election_id`, chiave pubblica elettore, nonce, timestamp.
Token	1018 byte	`VotingToken` in chiaro, misurato pre-cifratura: permit + firma GA.
Ballot	2447 byte	Token, ciphertext RSA scheda, commitment, firma elettore.
BoardRecord	∼1146 byte	`Mboard` + metadati catena append-only.
Receipt	684 byte	`leaf_hash`, `ballot_commit`, `encrypted_vote_hash`, firma ER.
MerkleProof	480–1048 byte	Cresce con $\log_2(n)$ del numero di record.

Il `Ballot` è il messaggio più pesante, circa 2.4 KB, soprattutto a causa del token e del ciphertext RSA della scheda. La `Receipt` è compatta, circa 684 byte, e non espone la preferenza.

6 Latenza di verifica

Nel benchmark sono misurate separatamente le seguenti fasi:

Fase di verifica	**Tempo medio**
Verifica firma, tutte le occorrenze	∼0.04–0.06 ms
Verifica token	< 0.001 ms
Verifica Merkle path	∼0.005–0.011 ms
Verifica ricevuta nel flusso client	Inclusa in `verify_signature`.

La verifica individuale, implementata tramite `verify_individual_inclusion`, combina:

1. verifica certificato ER e firma della receipt;
2. confronto tra `ballot_commit` e `encrypted_vote_hash`;
3. verifica Merkle path contro `root_final`.

Nel complesso, la verifica individuale è nell'ordine di decine di microsecondi per firma e path Merkle. È quindi trascurabile rispetto alla latenza end-to-end per voto, circa 50 ms, dove pesano maggiormente generazione chiavi, cifratura e I/O simulato.

7 Misurazioni effettuate

Il `MetricsCollector` registra tempi e dimensioni.

Tempi, tramite `measure` / `add_time`:

- `key_generation`, `sign`, `verify_signature`;
- `encrypt_vote`, `decrypt_vote`;
- `token_generation`, `token_verification`;
- `board_registration`, `merkle_build`, `merkle_verify_path`;
- `scrutiny`, `latency_per_vote` end-to-end per elettore.

Dimensioni, tramite `add_size` su JSON canonico:

- `AuthRequest`, `Token`, `Ballot`;
- `BoardRecord`, `Receipt`, `MerkleProof`.

Il benchmark esporta tutto in `benchmark_results.csv` e stampa una tabella riassuntiva con latenza, throughput, Merkle build e scrutinio.

8 Flusso protocollo

1. Setup GA: certificati VS/ER/SA/nodi + `ElectionParameters`.
2. VC → `AuthRequest` → GA, con IP + EV + VTI → `EncToken`.
3. VC → Ballot cifrato e firmato → SecureChannel → VS.
4. VS → verifica token, anti-riuso HMAC, commitment → `Mboard` → ER.
5. ER → `BoardRecord`, catena hash, + `Receipt` firmata.
6. Chiusura → `ER.finalize()` → Merkle root.
7. VC → `verify_individual_inclusion`, receipt + proof.
8. SA nodi → k autorizzazioni firmate.
9. SA → decifra, conta `YES` / `NO` → `TallyResult` firmato.
10. `verify_universal_result`: Merkle, conteggi, firma SA.

9 Limitazioni e funzionalità non implementate

9.1 Infrastruttura

- nessuna rete reale: no TLS, no latenza di rete, no DoS;
- nessun database persistente: append-only e `SpentTokensDB` solo in RAM;
- nessuna app mobile o UI web;
- nessuna blockchain.

9.2 Sicurezza e PKI

- certificati non X.509; CRL semplificata;
- canale sicuro implementato con Fernet simulato, non TLS con handshake reale;
- Identity Provider come lookup in un `set` locale, non SPID/eIDAS;
- coercizione non mitigata: screenshot, malware, osservazione fisica.

9.3 Crittografia avanzata

- no threshold decryption: dopo k firme di autorizzazione, un'unica chiave SA decifra tutto;
- no mix-net o anonimato formale oltre alla separazione token/voto;
- no zero-knowledge proofs di correttezza dello scrutinio;
- verifica universale: controlla firme, Merkle root e coerenza numerica, ma non prova pubblicamente che ogni scheda sia stata decifrata correttamente.

9.4 Operatività elettorale

- referendum binario fisso; validazione preferenza nel client commentata;
- un solo VS e un solo ER: no ridondanza né sharding;
- nessuna gestione concorrente reale su ER/VS, oltre al lock sul VTI;
- nessun meccanismo di recovery per transazioni `pending` in `SpentTokensDB`.

9.5 Test e robustezza

- test funzionali minimi tramite `tests.py`, non suite di sicurezza formale;
- il benchmark misura prestazioni locali, non scenari distribuiti o multi-processo.

10 Conclusione

La PoC dimostra in modo coerente il **flusso completo** del protocollo WP2: separazione dei ruoli, token monouso con lookup blindato, Bulletin Board verificabile con Merkle tree, ricevuta senza preferenza e scrutinio con quorum simulato. Le prestazioni su macchina locale sono buone per una simulazione, circa 20 voti/s, con verifiche nell'ordine dei microsecondi. Il collo di bottiglia principale è nella generazione RSA e, a scrutinio chiuso, nella decifratura scheda per scheda.

Resta un prototipo didattico: corretto sul flusso e sulle verifiche principali, ma non deployabile come sistema elettorale reale senza affrontare le limitazioni elencate.

La repository GitHub relativa a tale prototipo è visualizzabile al seguente link:
`https://github.com/gennaro24/APS-Project-Work.git`